arXiv:2607.13429v3 [cs.RO] 28 Oct 2026

GENERALIZABLE VLA FINETUNING VIA REPRESENTATION ANCHORING AND LANGUAGE-ACTION ALIGNMENT

Dwip Dalal[1], Shivansh Patel[1], Chahit Jain[1], Jeonghwan Kim[1], Utkarsh Mishra[2], Yuchen Song[3], Hyeonjeong Ha[1], Alex Baratian[1], Heng Ji[1], Svetlana Lazebnik[1*], Unnat Jain[3*]

[1]University of Illinois Urbana-Champaign [2]Texas A&M University
[3]University of California, Irvine

ABSTRACT

Finetuning a pretrained vision-language model (VLM) on robot demonstrations via behavior cloning (BC) has become the standard recipe for vision-language-action (VLA) policies. However, BC finetuning progressively overwrites the pretrained representations that support visual and semantic generalization. Co-training on web image-text data, a common remedy, applies language and action losses to separate observations, leaving VLAs with language-action misalignment that standard manipulation benchmarks do not expose. We propose Anchor-Align, which augments BC with two objectives: *Vision-Language Anchoring* distills layer-wise representations from a frozen VLM copy to prevent this drift, while *Language-Action Alignment* converts each action target into a discrete motion-direction label and jointly trains language and action prediction on the same robot observation. We conduct real-world evaluations across eight manipulation settings on single-arm xArm7 and bimanual YAM robots, using two VLA architectures with regression and flow-matching action heads. Across these settings, Anchor-Align consistently improves over BC on novel targets, layouts, and motion-sensitive bimanual tasks requiring coordinated control. At scale in simulation, we demonstrate consistent improvements on OOD perturbations, perceptual robustness, and long-horizon control across LIBERO-PRO, LIBERO-Plus, and CALVIN, respectively, suggesting that preserving pretrained representations and effective action learning are not fundamentally at odds. Project page: `https://anchoralignvla.github.io/`

1 INTRODUCTION

Vision-Language-Action (VLA) models adapt pretrained vision-language models (VLMs) into robot manipulation policies by finetuning on expert demonstrations (Zitkovich et al., 2023; Kim et al., 2024). Action learning typically uses behavior cloning (BC), with either direct regression (Kim et al., 2025) or flow matching and diffusion (Black et al., 2025b). The premise is that such finetuning should transfer the VLM's semantic priors (understanding of spatial layout, directions, color, shape, etc.) to the resulting control policy. Realizing this promise requires more than retaining pretrained knowledge: finetuning must make it useful for control.

Consider a VLA policy finetuned to "pick up the green mug and place it on the plate" in a scene containing both a green and pink mug (Fig. 1). Since the two mugs share the same shape and manipulation affordances, the policy should generalize to "pick up the pink mug" if finetuning preserves the VLM's color understanding. However, with existing methods, it does not. In fact, BC finetuning corrupts the very prior that makes VLMs worth adapting, leading to two failure modes that persist even under good training practices.

*Equal advising

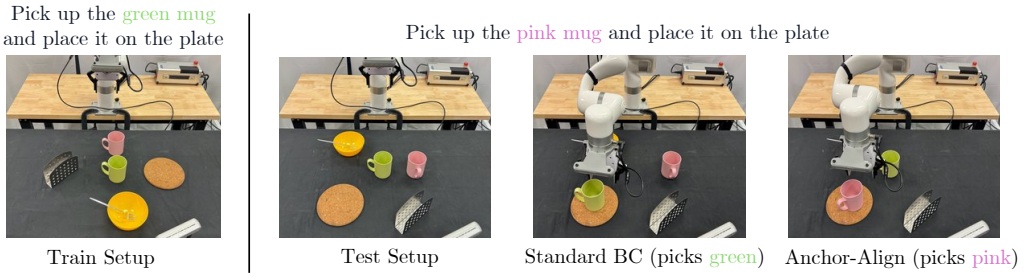


Figure 1: **Out-of-distribution test for VLAs.** Standard BC, finetuned to pick up the green mug, reaches for the green mug even when instructed to pick up the pink mug, whereas Anchor-Align retains the VLM's representations and successfully completes the task.

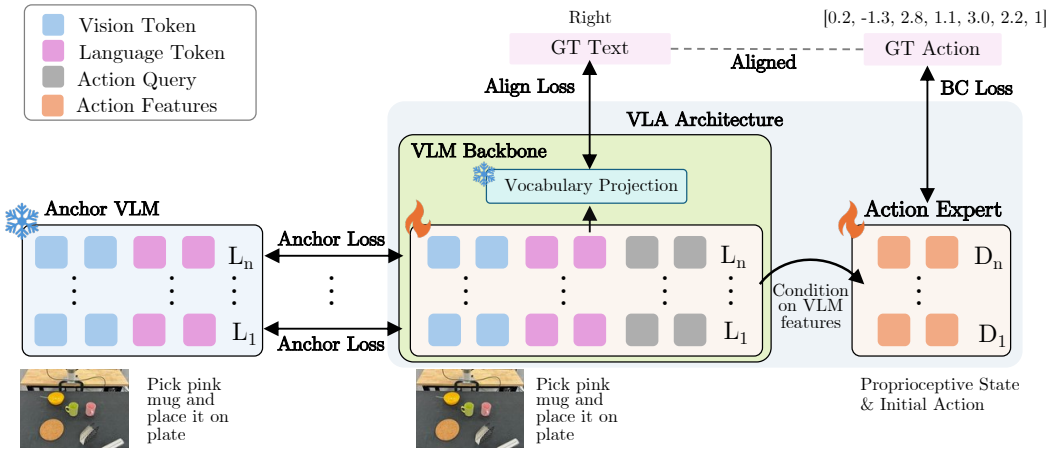


Figure 2: **Anchor-Align preserves VLM priors and aligns them with action during VLA finetuning.** Vision-Language Anchoring distills representations from a frozen pretrained VLM into the trainable VLA at every transformer layer (Anchor loss), preventing catastrophic forgetting. Language-Action Alignment programmatically converts each ground-truth demonstration trajectory into a discrete language label and trains the model to predict this label on the same observation it acts on (Align loss). Together, these objectives retain the pretrained representations while grounding action generation in VLM semantics.

First, standard BC optimizes only the action prediction loss, with nothing protecting the VLM's pretrained representations from being overwritten. Over the course of finetuning, these updates progressively erase the visuolinguistic and spatial concepts the VLM acquired during internet-scale pretraining. Concretely, on our physical xArm7 robot, a VLA trained to pick up the green mug reaches for the green mug 90% of the time even when instructed to pick up the pink one, indicating that color grounding has been erased from the backbone. On LIBERO-PRO's position-swap test, where object positions are rearranged at evaluation, the policy almost always replays its memorized training trajectory rather than acting on the current observation. To fix this failure mode, recent work has proposed co-training, or augmenting continuous action data with general-purpose VQA, captioning, and scene-description data (Lee et al., 2025; Driess et al., 2025; Yang et al., 2025; Zhou et al., 2025c) to prevent catastrophic forgetting. Yet we find that on the position-swap test, co-trained VLAs score 0%. Evidently, losses on such auxiliary data do not transfer the VLM's semantic understanding to the finetuned VLA. As a more direct remedy, we propose *Vision-Language Anchoring*, or maintaining a frozen copy of the pretrained VLM (Fig. 2, left) and applying a distillation loss throughout finetuning. This requires no additional data or architectural changes, yet recovers out-of-distribution (OOD) generalization.

The second failure mode runs even deeper. Co-training gives the VLA's two heads disjoint supervision: the action head is supervised by expert demonstrations while the language head is supervised by losses on generic image-text tasks. Because the heads share a backbone but are never supervised on the same robot observation, their predictions may contradict each other: when the action head predicts a movement to the right, the language head may predict "left." To fix this misalignment and further improve task success, we introduce *Language-Action Alignment*, which co-supervises both heads on the same robot observation during finetuning (Fig. 2, top). To supply the aligned supervi-

sory data, we automatically convert continuous action targets in expert demonstrations into discrete action directions expressed in language.

We propose combining Vision-Language Anchoring and Language-Action Alignment with the standard BC loss to form **Anchor-Align**, a VLA finetuning method with consistent improvements across simulation and real-world experiments under both regression and flow-matching action heads. In simulation, our method improves OOD generalization on LIBERO-PRO and LIBERO-Plus benchmarks, which introduce unseen spatial rearrangements, camera perturbations, and other shifts absent from training, as well as long-horizon control on CALVIN. In real-world experiments on an xArm7 robot, Anchor-Align improves spatial generalization, target generalization, and target grounding in clutter. On a bimanual YAM robot, it further improves motion-sensitive control, spanning reactive grasping and coordinated bimanual transport under visual shift.

2 Related Works

General-Purpose VLAs for Robot Manipulation. VLM-based policies leverage pretrained vision-language representations for instruction-conditioned control (Driess et al., 2023; Li et al., 2023; Qu et al., 2025b; Kim et al., 2025; Zitkovich et al., 2023). Recent general-purpose VLAs pair these pretrained backbones with expressive action-generation modules, including flow-matching action experts (Black et al., 2025b;a) and diffusion transformers (Bjorck et al., 2025). VLA-Adapter (Wang et al., 2025) enables direct adaptation without large-scale robotic pretraining through lightweight Bridge Attention, while StarVLA (StarVLA Community, 2026) provides a modular framework for combining backbones and action heads. We build on these two frameworks, finetuning pretrained VLMs directly on expert demonstrations.

Preserving VLM Representations in VLAs. Pretrained visual representations are a well-established driver of robot manipulation performance (Nair et al., 2022; Radosavovic et al., 2023; Dasari et al., 2023; Wang et al., 2024). In VLAs, this requirement extends beyond the vision encoder to the full VLM backbone, since semantic and spatial reasoning are distributed across both vision and language representations. Prior VLAs commonly mitigate catastrophic forgetting by co-training on robot and generic vision-language data (Yang et al., 2025; Lee et al., 2025; Qu et al., 2025a; Wen et al., 2025), with variants adding action-grounding objectives, perception tokens, or chain-of-thought plans (Yang et al., 2025; Lee et al., 2025; Zawalski et al., 2024; Zhou et al., 2025c;b). Grover et al. (2025) instead preserves pretrained features architecturally, anchoring a frozen vision encoder with a trainable one. MAPS (Huang et al., 2025) penalizes each module's weights for drifting from the pretrained checkpoint, constraining vision layers more tightly than action-oriented ones, and Kachaev et al. (2025) aligns visual patch features at a single intermediate layer with a frozen vision foundation model. Anchor-Align instead applies two simple losses at the VLM backbone level throughout finetuning, jointly preserving the pretrained VLM prior and enriching its representations with language supervision grounded in continuous robot actions. This couple's preservation with action-relevant adaptation, rather than treating preservation as an end goal.

Language supervision for action learning. Language supports action learning through intermediate predictions, action representations, and auxiliary supervision. Pretrained VLM knowledge has shown great support for planning and action selection in visual navigation (Dalal et al., 2026b). RT-H (Belkhale et al., 2024) conditions low-level actions on predicted language motions, while ECoT (Zawalski et al., 2024) and InSpire (Zhang et al., 2025) introduce grounded reasoning chains and object–gripper spatial relations as textual intermediates. VLA-0 (Goyal et al., 2025) serializes discretized actions as text, whereas Actions-as-Language (Hancock et al., 2025) expresses low-level commands in natural language within a reasoning hierarchy. LAP (Zha et al., 2026) supervises structured action-chunk descriptions; its LAP-3B implementation pairs this objective with a continuous-action expert whose gradients are blocked from the VLM. Anchor-Align instead preserves pretrained representations while adapting the VLM backbone through continuous-action gradients. Vision-Language Anchoring constrains representational drift, while Language-Action Alignment grounds language predictions in the same action targets supervised by BC. Language thus supplements continuous-action learning rather than replacing it, with both objectives using only existing demonstrations and requiring no intermediate language generation at inference.

3 ANCHOR-ALIGN METHOD

This section presents Anchor-Align (Fig. 2), a recipe that finetunes a pretrained VLM into a continuous action policy while preserving its pretrained VL representations. We first describe our base VLA architecture and training objective (Sec. 3.1), then the two objectives Anchor-Align adds on top of standard BC: *Vision-Language Anchoring* (Sec. 3.2) and *Language-Action Alignment* (Sec. 3.3).

3.1 BASE VLA ARCHITECTURE

We consider VLAs that adopt a pretrained vision-language backbone with a dedicated action head (StarVLA Community, 2026; Wang et al., 2025), as shown in Figure 2. The backbone encodes the visual observation and language instruction into a hierarchy of multimodal representations. Separately, robot proprioception is encoded by a lightweight state projector and provided to the action pathway as a conditioning signal. A fixed set of learned action queries extracts task-relevant information from the vision-language representations, either through joint self-attention within the backbone or through cross-attention in the action head. The action head then combines the resulting query features with the encoded robot state to predict a fixed-horizon chunk of continuous 7-DoF actions (3D translation and rotation offsets plus gripper control) using either direct regression or conditional flow matching.

We train this architecture with three objectives. The primary term is the standard BC loss $\mathcal{L}_{\text{action}}$ between the predicted and ground-truth action chunks, $\hat{\mathbf{a}}^{\text{cont}}$ and $\mathbf{a}^{\text{cont}}$ respectively. The form of the BC loss depends on the action-head architecture: an L_1 loss for a regression action head, and an L_2 loss over the predicted velocity for a flow-matching action head. To it we add two objectives, both evaluated on the same robot observation: an anchoring loss $\mathcal{L}_{\text{anchor}}$ that preserves the backbone's pretrained vision-language representations (Sec. 3.2), and an alignment loss $\mathcal{L}_{\text{align}}$ that grounds the model's language predictions in its actions (Sec. 3.3). The three terms are optimized jointly:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{action}} + \lambda_{\text{anchor}}\,\mathcal{L}_{\text{anchor}} + \lambda_{\text{align}}\,\mathcal{L}_{\text{align}}, \tag{1}$$

where λ_{anchor} and λ_{align} weight the two added objectives against the action loss.

3.2 VISION-LANGUAGE ANCHORING

To prevent catastrophic forgetting, we anchor the VLA's backbone VLM with a frozen copy of the same VLM (the *anchor* VLM), which processes the same input batch (images, text) in parallel. This distills the original pretrained VLM's representations into the trainable backbone. In effect, the anchor VLM serves as a frozen teacher, a role well established in the ML literature: feature distillation trains a student to match a teacher's representations (Hinton et al., 2015; Romero et al., 2015; Jiao et al., 2020), and anti-forgetting methods regularize a finetuned model toward its pretrained copy in weight space (Kirkpatrick et al., 2017; Li et al., 2018) or representation space (Li & Hoiem, 2018; Douillard et al., 2020; Mukhoti et al., 2024). We anchor hidden states of the vision and text tokens between the backbone and anchor VLM at every decoder layer $u \in \mathcal{D}$, where $\mathcal{D}$ is the set of decoder layers. Let $\mathbf{m}$ denote the set of vision and text positions. The anchoring loss is the mean squared error (MSE) between the two sets of hidden states, averaged over all $|\mathcal{D}|$ decoder layers:

$$\mathcal{L}_{\text{anchor}} = \frac{1}{|\mathcal{D}|}\sum_{u\in\mathcal{D}}\frac{1}{Nd}\sum_{i\in\mathbf{m}}\sum_{j=1}^{d}\left(\mathbf{H}_u^S[i,j] - \mathbf{H}_u^A[i,j]\right)^2, \tag{2}$$

where $\mathbf{H}_u^S$ and $\mathbf{H}_u^A$ are the hidden states of the trainable backbone (S) and the anchor (A) at decoder layer u, $\mathbf{H}_u[i,j]$ is the j-th feature of the token at position i, $N = |\mathbf{m}|$ is the number of vision and text tokens, and d is the hidden dimension of the VLM.

3.3 LANGUAGE-ACTION ALIGNMENT

As illustrated in Fig. 3, we programmatically convert the continuous action target $\mathbf{a}^{\text{cont}}$ into a motion-direction label $\mathbf{a}^{\text{lang}}$ for each non-stationary demonstration chunk and use it to supervise the VLM backbone on the same observation used for continuous action prediction. We use the last-layer hidden state of the last instruction token, immediately before action prediction, which summarizes the visual prefix and language instruction. With N vision and text tokens, this pre-action hidden state is $\mathbf{h}^{\text{pre}} = \mathbf{H}^S[N] \in \mathbb{R}^d$, where $\mathbf{H}^S$ denotes the backbone VLM's last-layer hidden states. We

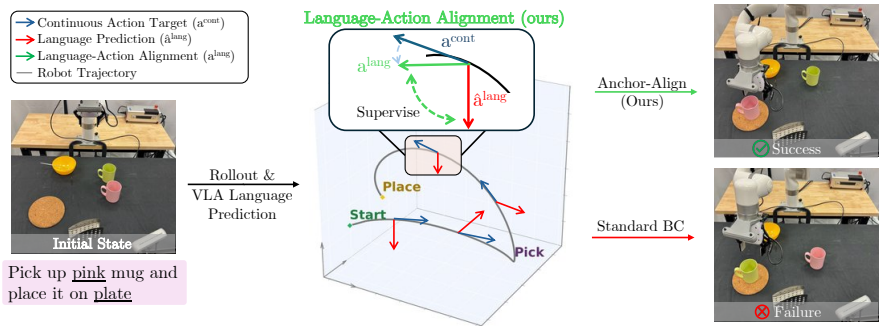


Figure 3: **Anchor-Align VLA aligns the VLM's language predictions with action generation during VLA finetuning.** Standard BC supervises only action prediction, allowing the model's language output (red) to contradict the required motion on the same observation (blue). Language-Action Alignment derives a discrete motion-direction label from each ground-truth action target and trains the backbone to predict it on the same observation (green), bringing language and action into agreement.

project $\mathbf{h}^{\text{pre}}$ onto the vocabulary through a learned projection and the frozen pretrained language head: $\hat{\mathbf{a}}^{\text{lang}} = \mathbf{W}_{\text{lm}}\mathbf{W}_{\text{proj}}\mathbf{h}^{\text{pre}} \in \mathbb{R}^{|\mathcal{V}|}$, where $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{d\times d}$ is learned and $\mathbf{W}_{\text{lm}} \in \mathbb{R}^{|\mathcal{V}|\times d}$ is the frozen pretrained LM head. The alignment loss is the cross-entropy between these vocabulary logits and the ground-truth direction label:

$$\mathcal{L}_{\text{align}} = \text{CE}(\hat{\mathbf{a}}^{\text{lang}}, \mathbf{a}^{\text{lang}}). \tag{3}$$

Language Label Construction. The alignment targets are derived from ground-truth action trajectories with no human annotation. For each training sample, we assign one of six interpretable direction labels, $W = \{up, down, left, right, forward, backward\}$, through three steps: average chunking, filtering, and discretization. Given the batch of action targets $\mathbf{A} \in \mathbb{R}^{B\times K\times 7}$ ($\mathbf{a}^{\text{cont}}$ stacked over B batch samples, each a K-step chunk), we average the translational components across the chunk dimension to reduce action noise: $\bar{\mathbf{v}} = \frac{1}{K}\sum_{k=1}^{K}\mathbf{A}_{:,k,1:3} \in \mathbb{R}^{B\times 3}$. We then remove near-stationary samples satisfying $\|\bar{\mathbf{v}}_i\|_2 < \tau$ (where i indexes samples in the batch), with τ a small stationarity threshold. For each remaining sample, we choose the dominant translation axis, $j^\star = \arg\max_{j\in\{x,y,z\}} |\bar{v}_{i,j}|$, where $\bar{v}_{i,j}$ is the j-th scalar component of $\bar{\mathbf{v}}_i$, and use the sign of $\bar{v}_{i,j^\star}$ to select the corresponding direction word. The resulting direction label $\mathbf{a}^{\text{lang}}$ supervises the language-alignment loss. Training implementation details are given in App. B.1; the diagnostic evaluation using these direction labels is described in App. E.1.

4 Experiments

We evaluate on simulation benchmarks and real-world robot experiments. Sec. 4.1 describes the benchmarks, baselines, and implementation details. Sec. 4.2 and Sec. 4.3 report results and ablations on LIBERO-PRO, LIBERO-Plus, and CALVIN. Sec. 4.4 results on an xArm7 and a bimanual YAM.

4.1 Experimental Setup

Benchmarks. We evaluate on LIBERO (Liu et al., 2023a), LIBERO-PRO (Zhou et al., 2025a), LIBERO-Plus (Fei et al., 2025), and CALVIN. LIBERO comprises four 10-task suites (Spatial, Object, Goal, Long) whose test setups follow the training distribution, probing only in-distribution performance and now being largely saturated. We therefore report standard LIBERO results in App. C.2, where Anchor-Align VLA outperforms all baselines, and focus below on the more challenging LIBERO-PRO and LIBERO-Plus stress tests. LIBERO-PRO (Zhou et al., 2025a) evaluates *semantic generalization* under OOD perturbations: language rephrase, object swap, and position swap. LIBERO-Plus (Fei et al., 2025) evaluates *perceptual robustness* across seven axes reflecting real-world deployment variability: camera viewpoint, lighting, background texture, object layout, robot initial state, language instruction, and sensor noise. Subcategory details are provided in App. C.1. Finally, CALVIN ABC→D tests long-horizon, language-conditioned manipulation by training on environments A, Bx, and C and evaluating zero-shot on held-out environment D. We further extend our study by collecting a dataset of more than 300 real-world demonstrations. We use this dataset to evaluate performance across 8 manipulation tasks.

Table 1: **Success rates on the LIBERO-PRO and LIBERO-Plus benchmarks.** Anchor-Align VLA achieves the highest mean on both benchmarks. The best result in each column is **bolded** and the second-best is underlined. The 5-seed mean and variance are reported in App. C.4, confirming that these gains are statistically significant.

Method	LIBERO-PRO				LIBERO-Plus							
	Lang. Reph.	Object Swap	Pos. Swap	Mean	Lang. Instr.	Bg. Text.	Robot Init	Cam. View	Obj. Layout	Light Cond.	Sensor Noise	Mean
MolmoAct (Lee et al., 2025)	77.8	82.4	0.0	53.4	79.5	84.1	47.4	10.1	76.5	77.4	53.4	59.9
VLA-0 (Goyal et al., 2025)	75.4	93.0	5.4	57.9	60.0	70.2	34.9	42.6	83.9	88.7	60.1	62.0
OpenVLA-OFT (Kim et al., 2025)	74.4	95.2	0.0	56.5	81.5	95.7	40.3	94.7	88.6	95.5	28.2	74.1
VLA-Adapter (Wang et al., 2025)	91.1	89.6	2.3	61.0	85.1	90.7	52.6	92.6	93.2	93.2	89.5	85.1
VLA-A [Frozen] (Wang et al., 2025)	56.0	73.4	0.0	43.1	41.5	70.9	35.1	94.4	62.3	84.9	36.2	59.9
Co-training + KI (Driess et al., 2025)	53.6	92.0	0.0	48.5	61.5	93.8	48.6	89.6	77.1	96.9	48.4	72.4
LAP (Zha et al., 2026)	93.2	95.0	12.4	66.9	**88.1**	99.2	54.0	72.6	92.5	95.5	76.1	81.8
MAPS (Huang et al., 2025)	93.0	90.2	11.6	64.9	87.1	97.3	52.9	95.2	87.5	97.3	90.6	86.3
Anchor-Align VLA	**97.0**	**96.2**	**22.6**	**71.9**	87.2	**99.6**	**59.1**	**96.3**	**97.4**	**99.0**	**96.9**	**90.3**

Baselines. We compare five baseline families (details in App. A): (i) the VLA models OpenVLA-OFT (Kim et al., 2025) and VLA-Adapter (Wang et al., 2025) (our base model); (ii) the representation-preserving baselines VLA-Adapter[Frozen] (Wang et al., 2025), which freezes the backbone, and MAPS (Huang et al., 2025), our reimplementation of module-wise proximity scheduling on VLA-Adapter; (iii) the co-trained VLAs: MolmoAct (Lee et al., 2025) and *Co-training + KI*, our reimplementation of Knowledge Insulation (Driess et al., 2025) on VLA-Adapter; (iv) the language-based action baselines VLA-0 (Goyal et al., 2025), which represents discretized actions as text, and LAP (Zha et al., 2026), which supervises language descriptions of action-chunk motion; and (v) the CALVIN baselines UniVLA (Bu et al., 2025) and OpenHelix (Cui et al., 2025).

Implementation Details. We use Prismatic-Qwen2.5-0.5B (Yang et al., 2024) as VLM backbone and finetune it with LoRA (Hu et al., 2022) with rank $r = 64$, applied on all layers. The input sequence is vision + text + proprioceptive state. We use a regression action head with bridge attention to layer-wise condition the action head latents on the VLM features (Wang et al., 2025). Image encoding, hyperparameters, and other implementation details are provided in App. B. We additionally evaluate StarVLA (StarVLA Community, 2026), which pairs Qwen2.5-VL (Bai et al., 2025) with a flow-matching GR00T FM-DiT (Bjorck et al., 2025) action head, in our real-world experiments.

4.2 Simulation Benchmark Comparison

Anchor-Align improves visual grounding and spatial understanding.

Tab. 1 shows that Anchor-Align VLA achieves the highest mean on both LIBERO-PRO and LIBERO-Plus. For LIBERO-PRO, the position-swap axis is the hardest: task-relevant objects are rearranged, so a policy that memorized fixed scene-to-action mappings fails. Most baselines fail almost entirely under this setting, while even the strongest alternatives reach only 12.4%; in contrast, Anchor-Align VLA achieves 22.6%. Fig. 4 shows qualitative examples from LIBERO-PRO, where Anchor-Align VLA generalizes to the perturbed scene, while standard BC replays its training trajectory.

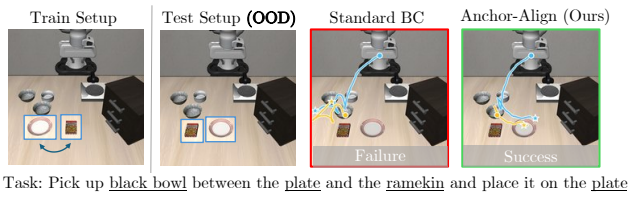


Figure 4: LIBERO-PRO Swap, performance of Anchor-Align in OOD condition.

Anchor-Align improves robustness to perturbations. LIBERO-Plus consists of challenging perturbations like changing background texture, adding sensor noise, or unseen initial robot arm configurations. As we can see from Tab. 1, Anchor-Align VLA improves over VLA-Adapter (Wang et al., 2025) on every axis, with the largest gains on the hardest perturbations: background texture (+8.9), sensor noise (+7.4), robot initial state (+6.5), and lighting (+5.8). In absolute terms, Anchor-Align VLA succeeds 99.6% under background texture changes and 99.0% under lighting changes. Per-suite robustness breakdowns for the LIBERO Long, Object, and Goal suites in App. C.3.

Anchor-Align improves long-horizon generalization. Tab. 2 shows results on the long-horizon CALVIN ABC→D benchmark. In each CALVIN rollout, the policy receives a chain of five language instructions and must complete them consecutively. The $k/5$ columns report the fraction of rollouts that complete the first k instructions, and **Len** denotes the average number of tasks completed consecutively (out of five). Because the chain advances only if each instruction succeeds, small grounding errors compound over the rollout. We therefore compare against the baselines most relevant to this claim: OpenVLA-OFT and UniVLA, large-scale action-pretrained VLAs; OpenHelix, a recent CALVIN-specific VLA; and VLA-Adapter trained with standard BC. Anchor-Align VLA is the best at every chain length and attains the longest average rollout, with the gain over VLA-Adapter widening on the deepest chains where errors compound, from $+0.8$ at one instruction to $+4.8$ at five. The results indicate preserving and aligning pretrained VLM representations improves long-horizon generalization.

Table 2: Long-horizon generalization on CALVIN ABC→D

CALVIN ABC→D	1/5 (↑)	2/5 (↑)	3/5 (↑)	4/5 (↑)	5/5 (↑)	Len (↑)
UniVLA (Bu et al., 2025)	95.5	85.8	75.4	66.9	56.5	3.8
OpenVLA-OFT (Kim et al., 2025)	96.3	89.1	82.4	75.8	66.5	4.1
OpenHelix (Cui et al., 2025)	97.1	91.4	82.8	72.6	64.1	4.1
VLA-Adapter (Wang et al., 2025)	98.3	94.0	87.5	80.0	73.1	4.3
Anchor-Align VLA	**99.1**	**95.8**	**90.6**	**84.7**	**77.9**	**4.5**

4.3 Impact of Anchoring and Alignment

Both anchoring and alignment are necessary. Tab. 3 presents an ablation analysis that isolates the impact of our proposed additional loss terms by comparing the VLA-Adapter baseline with Anchor VLA (only the layer-wise MSE distillation loss $\mathcal{L}_{\text{anchor}}$), Align VLA (only the language-action alignment loss $\mathcal{L}_{\text{align}}$), and the full Anchor-Align VLA. We can see that each of anchoring and alignment independently improve over standard BC on every axis, and combining them in Anchor-Align VLA exceeds either alone. Thus, the two play complementary roles: anchoring preserves the representational substrate that alignment leverages.

Table 3: Ablations on LIBERO-PRO and LIBERO-Plus.

Method		PRO	Plus
VLA-Adapter	*Standard BC*	61.0	85.1
Align VLA	*Alignment only*	65.9	88.6
Anchor VLA	*Anchoring only*	68.1	87.3
Anchor-Align VLA	*Full method*	**71.9**	**90.3**

Semantic alignment beyond auxiliary classification. We test whether the gains from Language-Action Alignment depend on the semantic correspondence between demonstrated motion and its language label, rather than simply supervising direction prediction. We retain the same training setup, including anchoring and the frozen language head, but change the target words assigned to the six motion directions. *Shuffle* applies a fixed permutation of the six direction words (e.g., forward → right), preserving directional vocabulary while breaking its correspondence to the demonstrated motion. *Scatter* assigns each direction a fixed word unrelated to motion (e.g., forward → purple). Both controls retain a one-to-one mapping between motion directions and target words, requiring the model to distinguish the same directions without semantically matching words to actions. As shown in Tab. 4, both perform substantially worse than the semantically matched formulation, indicating that direction prediction alone is insufficient and that the gains depend on the correct correspondence between motion and language (details in App. B.5). Linear probes further show improved motion-direction decoding from the backbone (App. D.5). Rollout diagnostics show higher language–action agreement for Anchor-Align, positively correlated with task success (App. E.2).

Table 4: Semantic-label controls on LIBERO-PRO and LIBERO-Plus.

Method		PRO	Plus
VLA-Adapter	*Standard BC*	61.0	85.1
Shuffle	*Control*	61.4	84.9
Scatter	*Control*	63.3	85.7
Anchor-Align VLA	*Full method*	**71.9**	**90.3**

Anchor-Align preserves VLM representations. For effective generalization, a VLA must be able to perform visual reasoning, i.e., understand objects, their semantics, and their attributes in a scene, an ability that is a hallmark of pretrained VLM representations. GQA (Hudson & Manning, 2019) tests this ability through compositional questions spanning object, attribute, and spatial understanding. Here, we quantify visual reasoning in standard BC and Anchor-Align, through GQA accuracy over the course of finetuning with the frozen VLM (same VLM used as backbone for VLA

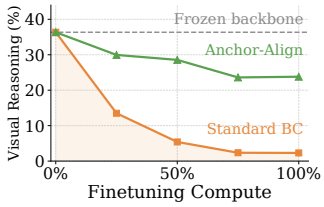


Figure 5: Visual reasoning accuracy during finetuning.

model) as an upper bound. As shown in Fig. 5, standard BC degrades monotonically, losing 94% of its GQA accuracy within 10K steps (catastrophic forgetting), whereas Anchor-Align retains roughly 65% of the pretrained accuracy. Correspondingly, LIBERO-PRO task success rates (Table 1) are 71.9% (Anchor-Align), 61.0% (standard BC), and 43.1% (frozen), showing that action performance and vision-language reasoning are not fundamentally at odds. We further substantiate this study by quantifying the change in representation by different finetuning paradigms in App. D.3.

4.4 Real-World Results

We extend our study to real-world robotic manipulation through controlled experiments across two distinct VLA architectures. We compare Anchor-Align against standard BC using VLA-Adapter (Wang et al., 2025), which pairs a Prismatic-Qwen2.5-0.5B backbone with a Bridge-Attention regression head, and StarVLA (StarVLA Community, 2026), which pairs a Qwen2.5-VL backbone with a GR00T FM-DiT flow-matching action head. These models span different VLM scales and action-generation architectures. We evaluate four capabilities: (1) target generalization, (2) spatial generalization, (3) target grounding in clutter, and (4) bimanual coordination.

We evaluate these capabilities across two robotic embodiments. The first three capabilities are evaluated on a single-arm 7-DoF UFactory xArm7, while bimanual coordination is evaluated on the 14-DoF bimanual YAM platform, where the tasks require reactive execution and coordinated motion across both arms. For each evaluated condition, we run 20 rollouts per method and report results as percentages. Each comparison uses the same underlying VLA architecture for Anchor-Align and standard BC, differing only in the training objective. Hardware, data collection, and deployment details are provided in App. B.4, with limitations and error analysis in Sec. 5.

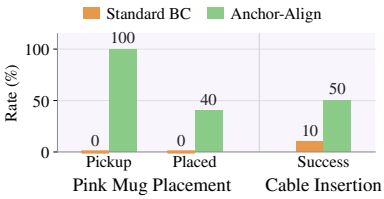
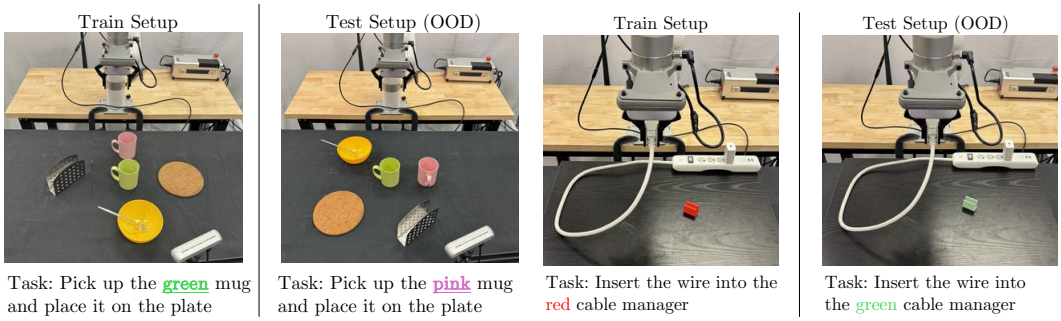


Figure 7: Results with StarVLA architecture.

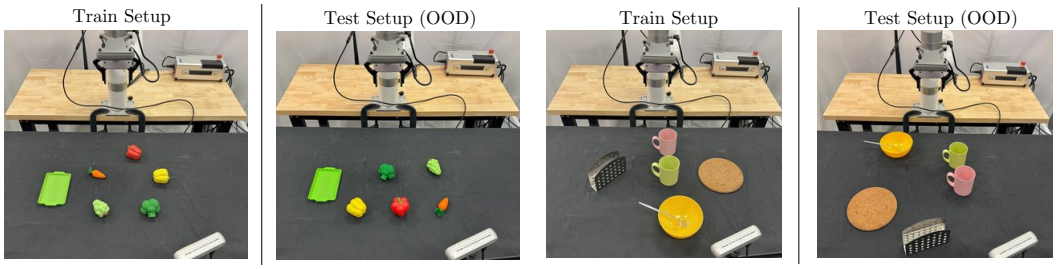
Target Generalization (OOD). We evaluate target generalization on the xArm7 across two tasks. 1) `Pink Mug Placement` tests an OOD change in the language-specified object together with an unseen spatial layout. During training, the policy receives the instruction *pick up the green mug and place it on the plate*; at test time, the instructed target changes to the *pink mug* and the objects are presented in a spatial configuration not seen during training (Fig. 6a). Successful execution therefore requires grounding the test-time language to the specified color and object while localizing that target under an unseen layout. 2) `Cable Insertion` evaluates an OOD change in the language-specified target in a high-precision manipulation task. During training, the policy is instructed to insert a wire into a red cable manager across five spatial configurations. At test time, the instruction specifies a green cable manager, which replaces the red one (Fig. 6b). Successful execution requires grounding the new language-specified target and precise control of end-effector position, orientation, and gripper actuation.

With StarVLA, standard BC continues to act on the training target in `Pink Mug Placement`: it picks up the green mug in 90% of trials. In contrast, Anchor-Align VLA picks up the instructed pink mug in 100% of trials and places it in 40%. The remaining failures occur during placement, which requires adapting the mug's trajectory to unseen plate locations while maintaining a stable



(a) `Pink Mug Placement`. (b) `Cable Insertion`

Figure 6: **Target Generalization (OOD) on xArm7.** **(a)** Target-object and spatial-layout changes between train & test. **(b)** Target-receptacle color shift changes between train & test.



(a) `Broccoli Placement`. (b) `Green Mug Placement`.

Figure 8: **Spatial Generalization (OOD). (a)** `Broccoli Placement` training and test setups. **(b)** `Green Mug Placement` training and test setups.

grasp. On `Cable Insertion`, Anchor-Align VLA achieves 50% success at inserting the wire into the instructed green cable manager, compared with 10% for standard BC (Fig. 7).

Spatial Generalization (OOD). We evaluate spatial generalization to unseen configurations across two tasks using VLA-Adapter and StarVLA. 1) `Broccoli Placement` varies the position and orientation of the broccoli and surrounding distractor objects relative to the training configurations (Fig. 8a). 2) `Green Mug Placement` introduces larger layout rearrangements, relocating the target and surrounding objects, including distractors, to configurations unseen during training, similar to the `Pink Mug Placement`. The instruction and target object remain unchanged: *pick up the green mug and place it on the plate* (Fig. 8b).

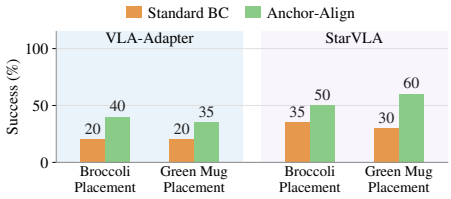


Figure 9: Success rates for `Broccoli Placement` and `Green Mug Placement`.

On `Broccoli Placement`, Anchor-Align VLA improves success from 20% to 40% with VLA-Adapter and from 35% to 50% with StarVLA. On `Green Mug Placement`, success increases from 20% to 35% with VLA-Adapter and from 30% to 60% with StarVLA (Fig. 9). These improvements across both architectures demonstrate better generalization to unseen spatial configurations while preserving the instruction and manipulation objective within each task.

Target Grounding in Clutter. We evaluate target grounding in clutter with `Clutter Scene` using both VLA-Adapter and StarVLA. The robot must pick up the object specified by the language instruction from among multiple distractors. We vary the target across three objects: `pineapple`, `yellow pepper`, and `red pepper`, requiring the policy to ground different instructions within the same scene. On `Clutter Scene`, Anchor-Align VLA increases success from 40% to 85% with VLA-Adapter and from 45% to 70% with StarVLA (Fig. 10). These gains show that Anchor-Align VLA more reliably identifies and manipulates the language-specified target despite surrounding distractors.

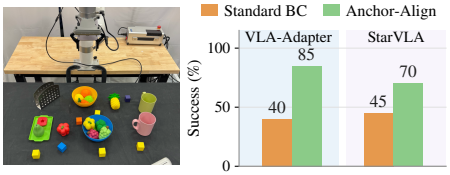


Figure 10: Left: Real-world evaluation setup. Right: Success rates for `Clutter Scene`.

Bimanual Coordination. In Anchor-Align, the *Language-Action Alignment* loss is applied independently to each arm, with a separate motion-direction label for each. We compare Anchor-Align against Standard BC across three tasks. 1) `Reactive Grasp` requires the robot to intercept and grasp a bottle carried by a moving person, adapting its reach and gripper closure to the bottle's motion. The person moves either from left to right or right to left, requiring the robot to switch its grasping arm based on the approach direction. Specifically, the left arm grasps the bottle when the person moves from right to left, and vice versa. The bottle varies in speed, height, and holding pose (Feng et al., 2026), requiring the policy to anticipate its motion rather than react only to its current position. 2) `Tomato Plate Transfer` requires the robot to lift a plate carrying eight tomatoes and place it onto a box without letting any tomatoes fall off. The robot must coordinate both arms to keep the plate level and maintain smooth motion, as differences in arm height can tilt the plate, while



(a) `Reactive Grasp`. (b) `Tomato Plate Transfer`. (c) `Tomato Plate Transfer (OOD)`. (d) Standard BC versus Anchor-Align on VLA-Adapter.

Figure 11: **Bimanual Coordination on YAM. (a)** `Reactive Grasp`: a person carries a bottle across the robot's workspace, requiring the robot to grasp it while in motion. **(b)** `Tomato Plate Transfer`: both arms lift a plate carrying eight tomatoes and place it onto a box without letting any tomatoes fall off. **(c)** `Tomato Plate Transfer (OOD)`: the same task in an unseen environment, with the box covered in white paper and the surrounding workspace walls removed. **(d)** Reactive Grasp success rate, plate-placement success rate (*Placed*), and mean percentage of tomatoes retained (*Retained*).

abrupt movements can cause the tomatoes to roll off. This task is challenging even for human teleoperators because both constraints must be maintained throughout the transfer (Song et al., 2026). 3) `Tomato Plate Transfer (OOD)` evaluates the same task in an unseen environment, where the box is covered with white paper and the surrounding workspace walls are removed.

With VLA-Adapter on YAM, Anchor-Align VLA improves `Reactive Grasp` success from 50% to 70%. On `Tomato Plate Transfer`, plate-placement success increases from 5% to 100%, while mean tomato retention increases from 6.3% to 60%. Under the unseen visual configuration, placement success increases from 0% to 70% and mean tomato retention from 0% to 47.5% (Fig. 11d). These gains indicate substantially improved motion-sensitive control, spanning both reactive grasping of moving targets and coordinated bimanual transport. To better understand the bimanual coordination gains, we analyze inter-arm motion in Appendix F.1, showing that Anchor-Align reduces the mean inter-arm height-change mismatch by 46.2%.

5 Limitations and Error Analysis

We manually inspect all failed xArm7 rollouts and bin clearly attributable failures into six modes (Fig. 12): *Incorrect Object*, approaches the wrong object; *Semantic Error*, selects a feature-similar distractor (e.g., green grapes instead of green broccoli); *Memorization Error*, ignores the scene and follows the training trajectory; *Grasp Failure*, reaches the target but does not descend to grasp; *Grasp-and-Drop Error*, grasps correctly but releases too early, too high, or not at all; and *Close Miss*, grounds correctly but misses by a small margin. Fig. 12 shows that Anchor-Align VLA eliminates semantic errors ($7 \rightarrow 0$), nearly eliminates incorrect-object approaches ($10 \rightarrow 1$), reduces memorization errors ($15 \rightarrow 9$), and nearly halves grasp failures ($15 \rightarrow 8$). The slight grasp-and-drop increase ($12 \rightarrow 13$) reflects more successful grasps, hence more opportunities for drop errors.

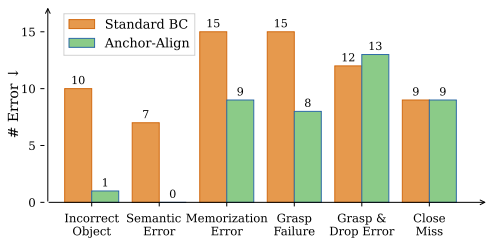


Figure 12: Failure mode analysis comparing standard BC and Anchor-Align.

6 Conclusion

Standard BC can overwrite pretrained VLM semantics, while co-training does not ensure language-action alignment. Anchor-Align addresses both through representation anchoring and language-action alignment losses, consistently improving OOD generalization, robustness, and real-world manipulation across VLA architectures and action heads without additional data or annotation. These results show that preserving and grounding VLM priors improves generalization and action learning.

ACKNOWLEDGMENTS

We thank R. Yeeshu Dhurandhar, Tongmiao Xu, Sagar Patil, and Zhenyang Feng for fruitful discussions. This research is based upon work supported by NSF grant CCF-2348624. This work also received support from the School of ICS at UC Irvine and from Modal through cloud compute credits and infrastructure. This research also used the DeltaAI advanced computing and data resource, which is supported by the National Science Foundation under award OAC 2320345 and the State of Illinois. DeltaAI is a joint effort of the University of Illinois Urbana-Champaign and its National Center for Supercomputing Applications. This work was also supported by the U.S. DARPA ECOLE Program under Award No. HR00112390060. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies, either expressed or implied, of DARPA or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for governmental purposes, notwithstanding any copyright annotation therein.

REFERENCES

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923*, 2025.

Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quan Vuong, Jonathan Tompson, Yevgen Chebotar, Debidatta Dwibedi, and Dorsa Sadigh. RT-H: Action Hierarchies Using Language. *arXiv preprint arXiv:2403.01823*, 2024.

Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. *arXiv preprint arXiv:2503.14734*, 2025.

Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pp. 17–40. PMLR, 2025a. URL `https://proceedings.mlr.press/v305/black25a.html`.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, Laura Smith, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A Vision-Language-Action Flow Model for General Robot Control. In *Robotics: Science and Systems*, 2025b.

Yiqing Bo, Ansh Soni, Sudhanshu Srivastava, and Meenakshi Khosla. Evaluating representational similarity measures from the lens of functional correspondence. *arXiv preprint arXiv:2411.14633*, 2024.

Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. UniVLA: Learning to Act Anywhere with Task-centric Latent Actions. *arXiv preprint arXiv:2505.06111*, 2025.

Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Khalil Meftah, Maxime Ellerbach, Jess Moss, and Thomas Wolf. LeRobot: State-of-the-art Machine Learning for Real-World Robotics in Pytorch. https://github.com/huggingface/lerobot, 2024.

Boyuan Chen, Zhuo Xu, Sean Kirmani, Brian Ichter, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. SpatialVLM: Endowing Vision-Language Models with Spatial Reasoning Capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14455–14465, June 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Chen_SpatialVLM_Endowing_Vision-Language_Models_with_Spatial_Reasoning_Capabilities_CVPR_2024_paper.html.

Can Cui, Pengxiang Ding, Wenxuan Song, Shuanghao Bai, Xinyang Tong, Zirui Ge, Runze Suo, Wanqi Zhou, Yang Liu, Bofang Jia, Han Zhao, Siteng Huang, and Donglin Wang. OpenHelix: A Short Survey, Empirical Analysis, and Open-Source Dual-System VLA Model for Robotic Manipulation. *arXiv preprint arXiv:2505.03912*, 2025.

Dwip Dalal, Madhav Kanda, Zhenhailong Wang, Heng Ji, and Unnat Jain. Compositional Reasoning via Joint Image and Language Decomposition. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Findings of the Association for Computational Linguistics: EACL 2026*, pp. 5753–5775, Rabat, Morocco, March 2026a. Association for Computational Linguistics. ISBN 979-8-89176-386-9. doi: 10.18653/v1/2026.findings-eacl.304. URL https://aclanthology.org/2026.findings-eacl.304/.

Dwip Dalal, Utkarsh Mishra, Narendra Ahuja, and Nebojsa Jojic. Can MLLMs Find Their Way in a City? Exploring Emergent Navigation from Web-Scale Knowledge. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8279–8303, Rabat, Morocco, March 2026b. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.387. URL https://aclanthology.org/2026.eacl-long.387/.

Dwip Dalal, Gautam Vashishtha, Utkarsh Mishra, Jeonghwan Kim, Madhav Kanda, Hyeonjeong Ha, Svetlana Lazebnik, Heng Ji, and Unnat Jain. Constructive Distortion: Improving MLLMs with Attention-Guided Image Warping. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 23639–23672, 2026c. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/288b63aa98084366c4536ba0574a0f22-Paper-Conference.pdf.

Sudeep Dasari, Mohan Kumar Srirama, Unnat Jain, and Abhinav Gupta. An Unbiased Look at Datasets for Visuo-Motor Pre-Training. In *Conference on Robot Learning*, pp. 1183–1198, 2023.

Arthur Douillard, Matthieu Cord, Charles Ollion, Thomas Robert, and Eduardo Valle. PODNet: Pooled Outputs Distillation for Small-Tasks Incremental Learning. In *European Conference on Computer Vision*, 2020.

Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. PaLM-E: An Embodied Multimodal Language Model. *arXiv preprint arXiv:2303.03378*, 2023.

Danny Driess, Jost Tobias Springenberg, Brian Ichter, Lili Yu, Adrian Li-Bell, Karl Pertsch, Allen Z. Ren, Homer Walke, Quan Vuong, Lucy Xiaoyang Shi, and Sergey Levine. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. *arXiv preprint arXiv:2505.23705*, 2025.

Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. LIBERO-Plus: In-depth Robustness Analysis of Vision-Language-Action Models. *arXiv preprint arXiv:2510.13626*, 2025.

Zhenyang Feng, Jimin Heo, Erik B. Sudderth, and Unnat Jain. TEMPO: Learning Temporal Context for Dynamic Robot Manipulation. *arXiv preprint arXiv:2609.16864*, 2026.

Ankit Goyal, Hugo Hadfield, Xuning Yang, Valts Blukis, and Fabio Ramos. VLA-0: Building State-of-the-Art VLAs with Zero Modification. *arXiv preprint arXiv:2510.13054*, 2025.

Arthur Gretton, Olivier Bousquet, Alex Smola, and Bernhard Schölkopf. Measuring Statistical Dependence with Hilbert–Schmidt Norms. In *Int. Conf. Algorithmic Learn. Theory*, pp. 63–77. Springer, 2005.

Shresth Grover, Akshay Gopalkrishnan, Bo Ai, Henrik I. Christensen, Hao Su, and Xuanlin Li. Enhancing Generalization in Vision-Language-Action Models by Preserving Pretrained Representations. *arXiv preprint arXiv:2509.11417*, 2025.

Asher J. Hancock, Xindi Wu, Lihan Zha, Olga Russakovsky, and Anirudha Majumdar. Actions as Language: Fine-Tuning VLMs into VLAs Without Catastrophic Forgetting. *arXiv preprint arXiv:2509.22195*, 2025. URL `https://arxiv.org/abs/2509.22195`.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*, 2015.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*, 2022.

Chengyue Huang, Mellon M. Zhang, Robert Azarcon, Glen Chou, and Zsolt Kira. MAPS: Preserving Vision-Language Representations via Module-Wise Proximity Scheduling for Better Vision-Language-Action Generalization. *arXiv preprint arXiv:2511.19878*, 2025.

Drew A. Hudson and Christopher D. Manning. GQA: A New Dataset for Real-World Visual Reasoning and Compositional Question Answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6700–6709, 2019.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. TinyBERT: Distilling BERT for Natural Language Understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.

Nikita Kachaev, Mikhail Kolosov, Daniil Zelezetsky, Alexey K Kovalev, and Aleksandr I Panov. Don't Blind Your VLA: Aligning Visual Representations for OOD Generalization. *arXiv preprint arXiv:2510.25616*, 2025.

Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An Open-Source Vision-Language-Action Model. In *Conference on Robot Learning*, 2024.

Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success. *arXiv preprint arXiv:2502.19645*, 2025.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of Neural Network Representations Revisited. In *Int. Conf. Mach. Learn.*, pp. 3519–3529, 2019.

Jason Lee, Jiafei Duan, Haoquan Fang, Yuquan Deng, Shuo Liu, Boyang Li, Bohan Fang, Jieyu Zhang, Yi Ru Wang, Sangho Lee, Winson Han, Wilbert Pumacay, Angelica Wu, Rose Hendrix, Karen Farley, Eli VanderBilt, Ali Farhadi, Dieter Fox, and Ranjay Krishna. MolmoAct: Action Reasoning Models that can Reason in Space. *arXiv preprint arXiv:2508.07917*, 2025.

Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, Hang Li, and Tao Kong. Vision-language foundation models as effective robot imitators. *arXiv preprint arXiv:2311.01378*, 2023.

Xuhong Li, Yves Grandvalet, and Franck Davoine. Explicit Inductive Bias for Transfer Learning with Convolutional Networks. In *International Conference on Machine Learning*, 2018.

Zhizhong Li and Derek Hoiem. Learning without Forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, 2018.

Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common Objects in Context. In *European Conference on Computer Vision*, pp. 740–755. Springer, 2014.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. In *NeurIPS*, 2023a.

Fangyu Liu, Guy Emerson, and Nigel Collier. Visual Spatial Reasoning. *Transactions of the Association for Computational Linguistics*, 11, 2023b.

Jishnu Mukhoti, Yarin Gal, Philip H. S. Torr, and Puneet K. Dokania. Fine-tuning can cripple your foundation model; preserving features may be the solution. *Transactions on Machine Learning Research*, 2024.

Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In *Conference on Robot Learning*, 2022.

Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning Robust Visual Features without Supervision. *Transactions on Machine Learning Research*, 2024.

Delin Qu, Haoming Song, Qizhi Chen, Zhaoqing Chen, Xianqiang Gao, Dong Wang, Xinyi Ye, Qi Lv, Modi Shi, Guanghui Ren, Cheng Ruan, Maoqing Yao, Haoran Yang, Jiacheng Bao, Bin Zhao, and Xuelong Li. EO-1: An Open Unified Embodied Foundation Model for General Robot Control. *arXiv preprint arXiv:2508.21112*, 2025a.

Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, and Xuelong Li. SpatialVLA: Exploring Spatial Representations for Visual-Language-Action Model. *arXiv preprint arXiv:2501.15830*, 2025b.

Ilija Radosavovic, Tete Xiao, Stephen James, Pieter Abbeel, Jitendra Malik, and Trevor Darrell. Real-World Robot Learning with Masked Visual Pre-training. In *Conference on Robot Learning*, pp. 416–426, 2023.

Sabela Ramos, Sertan Girgin, Léonard Hussenot, Damien Vincent, Hanna Yakubovich, Daniel Toyama, Anita Gergely, Piotr Stanczyk, Raphael Marinier, Jeremiah Harmsen, Olivier Pietquin, and Nikola Momchev. RLDS: an Ecosystem to Generate, Share and Use Datasets in Reinforcement Learning. *arXiv preprint arXiv:2111.02767*, 2021.

Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. FitNets: Hints for Thin Deep Nets. In *International Conference on Learning Representations*, 2015.

Chan Hee Song, Valts Blukis, Jonathan Tremblay, Stephen Tyree, Yu Su, and Stan Birchfield. RoboSpatial: Teaching Spatial Understanding to 2D and 3D Vision-Language Models for Robotics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15768–15780, June 2025. URL https://openaccess.thecvf.com/content/CVPR2025/html/Song_RoboSpatial_Teaching_Spatial_Understanding_to_2D_and_3D_Vision-Language_Models_CVPR_2025_paper.html.

Yuchen Song, Aditya Mittal, and Unnat Jain. Learning beyond what humans can demonstrate. *arXiv preprint arXiv:2609.24996*, 2026.

StarVLA Community. StarVLA: A Lego-like Codebase for Vision-Language-Action Model Developing. *arXiv preprint arXiv:2604.05014*, 2026.

Lirui Wang, Xinlei Chen, Jialiang Zhao, and Kaiming He. Scaling Proprioceptive-Visual Learning with Heterogeneous Pre-trained Transformers. In *Advances in Neural Information Processing Systems*, 2024.

Yihao Wang, Pengxiang Ding, Lingxiao Li, Can Cui, Zirui Ge, Xinyang Tong, Wenxuan Song, Han Zhao, Wei Zhao, Pengxu Hou, Siteng Huang, Yifan Tang, Wenhui Wang, Ru Zhang, Jianyi Liu, and Donglin Wang. VLA-Adapter: An Effective Paradigm for Tiny-Scale Vision-Language-Action Model. *arXiv preprint arXiv:2509.09372*, 2025.

Junjie Wen, Yichen Zhu, Jinming Li, Zhibin Tang, Chaomin Shen, and Feifei Feng. DexVLA: Vision-Language Model with Plug-In Diffusion Expert for General Robot Control. *arXiv preprint arXiv:2502.05855*, 2025.

Philipp Wu, Yide Shentu, Zhongke Yi, Xingyu Lin, and Pieter Abbeel. GELLO: A General, Low-Cost, and Intuitive Teleoperation Framework for Robot Manipulators. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 12156–12163. IEEE, 2024.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024.

Jianwei Yang, Reuben Tan, Qianhui Wu, Ruijie Zheng, Baolin Peng, Yongyuan Liang, Yu Gu, Mu Cai, Seonghyeon Ye, Joel Jang, Yuquan Deng, and Jianfeng Gao. Magma: A Foundation Model for Multimodal AI Agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 14203–14214, 2025.

Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. *arXiv preprint arXiv:2407.08693*, 2024.

Lihan Zha, Asher J. Hancock, Mingtong Zhang, Tenny Yin, Yixuan Huang, Dhruv Shah, Allen Z. Ren, and Anirudha Majumdar. LAP: Language-Action Pre-Training Enables Zero-shot Cross-Embodiment Transfer. *arXiv preprint arXiv:2602.10556*, 2026. URL `https://arxiv.org/abs/2602.10556`.

Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 11975–11986, 2023.

Ji Zhang, Shihan Wu, Xu Luo, Hao Wu, Lianli Gao, Heng Tao Shen, and Jingkuan Song. InSpire: Vision-Language-Action Models with Intrinsic Spatial Reasoning. *arXiv preprint arXiv:2505.13888*, 2025. URL `https://arxiv.org/abs/2505.13888`.

Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. LIBERO-PRO: Towards Robust and Fair Evaluation of Vision-Language-Action Models Beyond Memorization. *arXiv preprint arXiv:2510.03827*, 2025a.

Zhongyi Zhou, Yichen Zhu, Junjie Wen, Chaomin Shen, and Yi Xu. ChatVLA-2: Vision-Language-Action Model with Open-World Embodied Reasoning from Pretrained Knowledge. *arXiv preprint arXiv:2505.21906*, 2025b.

Zhongyi Zhou, Yichen Zhu, Minjie Zhu, Junjie Wen, Ning Liu, Zhiyuan Xu, Weibin Meng, Ran Cheng, Yaxin Peng, Chaomin Shen, and Feifei Feng. ChatVLA: Unified Multimodal Understanding and Robot Control with Vision-Language-Action Model. *arXiv preprint arXiv:2502.14420*, 2025c.

Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. In *Proceedings of the 7th Conference on Robot Learning (CoRL)*, volume 229 of *Proceedings of Machine Learning Research*, pp. 2165–2183. PMLR, 2023.

Appendix

A: Baseline Details 17
B: Implementation Details 18
B.1 Language-Action Alignment Head
B.2 Training Setup
B.3 Training Cost Comparison
B.4 Real-World Implementation
B.5 Alignment vs. Regularization: Shuffle and Scatter Controls
C: Extended Quantitative Results 20
C.1 LIBERO-PRO and LIBERO-Plus Subcategories
C.2 Success Rates on Standard LIBERO Suites
C.3 Per-Suite Robustness Breakdowns
C.4 Multi-Seed Evaluation: Statistical Significance
C.5 Backbone Regularization Controls
D: Representation Analysis: Language Preservation and Action Decodability 22
D.1 Centered Kernel Alignment (CKA)
D.2 Action Decodability (Linear Probing R^2)
D.3 Language Preservation and Action Decodability
D.4 Layer-wise Language Preservation
D.5 Direction Understanding in the Backbone
E: Language-Action Alignment Diagnostic 26
E.1 Diagnostic Protocol
E.2 Relationship to Task Success
F: Real-World Rollouts 27
F.1 Bimanual Height Coordination
F.2 Object-Orientation Perturbation
F.3 Anchor-Align Real-World Rollouts
G: Qualitative Results in LIBERO Simulator 32

A Baseline Details

This appendix extends the condensed baseline list in Sec. 4.1: baselines already introduced there are referenced briefly, and we detail the additional baselines that appear only in the appendix tables.

• **Standard VLM-based VLAs.** OpenVLA-OFT (Kim et al., 2025) and VLA-Adapter (Wang et al., 2025) (our base model) are introduced in Sec. 4.1. The standard-LIBERO comparison (Tab. 6) additionally includes VLA-0 (Goyal et al., 2025) (plain-text action integers).

• **Knowledge-preservation approaches.** $\pi_{0.5}$-KI (Driess et al., 2025) addresses catastrophic forgetting by training discrete FAST action tokens on the VLM backbone while stop-gradient prevents action gradients from corrupting pretrained representations. VLA-Adapter[Frozen] (Wang et al., 2025) represents the extreme preservation baseline: the VLM backbone is entirely frozen and only the action head is trained, trivially preserving all pretrained knowledge at the cost of reduced task performance. MAPS (Huang et al., 2025) instead keeps each module's weights close to the pretrained checkpoint through a layer-wise proximity schedule that constrains vision layers more tightly than action-oriented ones. Our MAPS baseline runs the authors' released proximity optimizer and schedule on VLA-Adapter with the maximum proximity weight of 0.5 that the MAPS authors use for VLA-Adapter on LIBERO; all other settings follow the standard BC recipe in App. B.2, and we evaluate the final 10K-step checkpoint.

• **Co-training baselines.** We compare *MolmoAct* and *Co-training + KI*, introduced in Sec. 4.1. MolmoAct (Lee et al., 2025) is an example of co-training with general-purpose web examples separate from robot demonstrations. Its robot-side supervision jointly models depth-perception tokens, image-space trajectory traces, and discretized actions from the same observations for 3D spatial reasoning. *Co-training + KI* is our reimplementation of Knowledge Insulation (Driess et al., 2025) on VLA-Adapter. We sample web VQA and robot action data at a ratio of 1:3 (web:action). Action finetuning uses LoRA: the language modeling head receives VQA supervision on web examples and FAST-tokenized action supervision on robot observations, while an L1 regression action head predicts continuous actions. Knowledge insulation blocks gradients from the continuous action head

into the backbone. These comparisons test whether co-training, including action-token supervision and knowledge insulation, is sufficient to preserve pretrained semantics and produce consistent language and action predictions.

Co-training during task-specific finetuning. Co-training has shown strong results in large-scale VLA pretraining (Lee et al., 2025), and Knowledge Insulation improves knowledge transfer during VLA training (Driess et al., 2025). Our VLM-to-VLA finetuning setting emphasizes learning fine-grained task-specific behavior. We hypothesize that general-purpose web supervision can act as an auxiliary objective with limited relevance to these task details, potentially competing with task-specific adaptation. This may help explain why our Co-training + KI baseline underperforms standard BC on mean LIBERO-PRO and LIBERO-Plus success (Tab. 1).

• **Long-horizon CALVIN baselines.** The main-paper CALVIN ABC→D comparison (Tab. 2) includes OpenVLA-OFT (Kim et al., 2025), UniVLA (Bu et al., 2025), OpenHelix (Cui et al., 2025), and VLA-Adapter (Wang et al., 2025). These baselines evaluate long-horizon, instruction-conditioned manipulation under environment shift.

B Implementation Details

For better reproducibility of our results, we include the full architectural specification, hyperparameter settings, and training cost breakdown below.

B.1 Language-Action Alignment Head

The alignment objective (Sec. 3.3) introduces a single learned linear projection $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{d\times d}$ with $d = 896$ (the hidden dimension of Qwen2.5-0.5B), adding $896^2 + 896 = 803{,}712$ parameters (1.5 MB in `bfloat16`). At each training step, we take the last-layer hidden state $\mathbf{h}_i^{\text{pre}} = \mathbf{H}^S[N] \in \mathbb{R}^d$ of the last of the N vision and text tokens, the position immediately before the first action token (located in code by scanning the label tensor for the first non-masked entry, which marks the start of the action tokens). It is then mapped through $\mathbf{W}_{\text{proj}}$ and the frozen pretrained language modeling head $\mathbf{W}_{\text{lm}} \in \mathbb{R}^{|\mathcal{V}|\times d}$ to produce the language logits $\hat{\mathbf{a}}_i^{\text{lang}} \in \mathbb{R}^{|\mathcal{V}|}$ over the full vocabulary ($|\mathcal{V}| = 151{,}646$). A cross-entropy loss is then computed between $\hat{\mathbf{a}}_i^{\text{lang}}$ and the ground-truth direction label $\mathbf{a}_i^{\text{lang}}$ (one of *forward*, *backward*, *left*, *right*, *up*, *down*), with near-stationary samples ($\|\bar{\mathbf{v}}_i\|_2 < \tau$, $\tau = 0.15$) masked via `ignore index`. Because W_{lm} is frozen, the alignment gradient flows only through W_{proj} and the backbone's LoRA parameters; the pretrained vocabulary head therefore remains fixed throughout alignment training. The projection head is the only additional trainable module beyond the LoRA adapters and action head already present in VLA-Adapter.

B.2 Training Setup

All models are trained on 4 NVIDIA GH200 GPUs using `bfloat16` mixed precision with PyTorch 2.7.

Hyperparameters. To ensure reproducibility, we report the complete training configuration used for the results in Tab. 1. The backbone VLM receives two camera images (third-person and wrist) along with the proprioceptive state as input. We use LoRA-based parameter-efficient finetuning with rank r=64 and scaling factor α=128, applied to all linear layers in the backbone, and merge the LoRA weights into the base model periodically during training. Training runs for 10,000 gradient steps with a batch size of 32, a learning rate of 2×10^{-4}, and standard image augmentation (random crops and color jitter). Checkpoints are saved every 2,500 steps. For the anchoring objective, we apply layer-wise MSE distillation ($\mathcal{L}_{\text{anchor}}$) across all 24 transformer layers with anchoring weight λ_{anchor}=0.05. For the alignment objective ($\mathcal{L}_{\text{align}}$), we use an alignment weight of λ_{align}=0.02 and a motion-magnitude threshold τ=0.15 to filter near-stationary samples. All remaining hyperparameters, including optimizer settings and data preprocessing, follow the recommended defaults from (Wang et al., 2025) for the baseline setup.

Image encoding. The 512 vision patches are obtained from two images: we extract DINOv2 (Oquab et al., 2024) and SigLIP (Zhai et al., 2023) features for each image and concatenate them along the feature dimension per patch, yielding 256 patches per image and a 512-patch vision prefix in total.

B.3 Training Cost Comparison

A key practical question is how much overhead knowledge-preservation methods add to standard action finetuning. On identical hardware, we compare Anchor-Align against **Standard BC** (VLA-Adapter (Wang et al., 2025)) and **Co-training + KI** (Driess et al., 2025), both detailed in Sec. 4.1. Co-training is the natural point of comparison because it explicitly targets the same goal as our method: retaining the VLM's pretrained visuolinguistic priors during finetuning. Tab. 5 summarizes the cost and performance trade-offs.

Table 5: Training cost and downstream performance comparison. The recorded profiling measurements show $3.4\times$ less overhead per step and $7\times$ less additional GPU memory for Anchor-Align, with no external data. KI timing and memory measurements are from the earlier profiling run; its success rate is taken from Tab. 1.

	Standard BC	**Co-training + KI**[1]	**Anchor-Align**
Wall-clock (s/it)	1.28	2.50	1.64
Overhead vs. baseline	—	+95%	+28%
2,500 steps (min)	53	104	68
Extra GPU memory	—	+5 GB	+0.7 GB
External data required	None	25K VQA	None
LIBERO-Plus Overall	85.1	72.4	**90.3**

In the earlier profiling run, Co-training + KI adds a full VQA forward and backward pass (with batch size 16) at every training step, incurring $+95\%$ wall-clock overhead and $+5$ GB of GPU memory for the additional activations and gradients. It further requires curating 25K external VQA samples (VSR (Liu et al., 2023b), GQA (Hudson & Manning, 2019), COCO (Lin et al., 2014)) and maintaining the language modeling head alongside the action head during training. Co-training + KI scores 53.6% on LIBERO-PRO language rephrase and 72.4% LIBERO-Plus overall, below standard BC (91.1% and 85.1%) in these evaluations.

In contrast, Anchor-Align adds only a single inference-only forward pass through a frozen copy of the pretrained VLM (the *anchor*). We replace the anchor's language modeling head ($151{,}646 \times 896 \approx 136$M parameters) with `nn.Identity()`, since only intermediate hidden states are needed for the layer-wise MSE loss, reducing the anchor's footprint to 0.7 GB. This yields $+28\%$ wall-clock overhead (0.36 s per step), requires no external data or annotation, and completes a full 2,500-step run in 68 minutes—36 minutes faster than co-training. Critically, Anchor-Align achieves 97.0% on language rephrase and 90.3% on LIBERO-Plus overall, demonstrating that preserving the VLM's representational geometry via anchoring is both cheaper and dramatically more effective than co-training on external VQA data. For larger backbone VLMs, the relative overhead would decrease further, since the anchor's inference-only forward pass scales sublinearly compared to the training backward pass.

B.4 Real-World Implementation

Throughout our real-world experiments, "standard BC" denotes the published VLA-Adapter (Wang et al., 2025) and StarVLA (StarVLA Community, 2026) architectures trained with their respective behavior-cloning objectives, using the same demonstrations and optimization settings as Anchor-Align VLA within each architecture.

xArm7 Setup. We uses xArm7 UFactory equipped with a wrist-mounted RGB camera and an external front-view camera, evaluated on tabletop tasks requiring object discrimination, spatial reasoning, and instruction-conditioned target selection under clutter. We collect 50 demonstration trajectories per task using a 3D-printed GELLO teleoperation device (Wu et al., 2024) with velocity control at 30 Hz; trajectories are stored in the LeRobot (Cadene et al., 2024) format and converted to RLDS (Ramos et al., 2021) for our training pipeline. Both baseline and Anchor-Align VLA are

[1] Our reimplementation on VLA-Adapter (Sec. 4.1).

trained with identical hyperparameters: VLA-Adapter policies for 10K steps and StarVLA policies for 50K steps. Evaluation follows the task definitions and rollout protocol in Sec. 4.4. At deployment, the low-level velocity controller runs at 100 Hz while the learned policy predicts actions at 30 Hz; commands are interpolated and velocity-clipped for safe execution.

Bimanual Robot Setup. We conduct our bimanual experiments on an I2RT YAM Ultra platform comprising two 6-DoF arms, each equipped with a 1-DoF parallel-jaw gripper. We collect demonstrations through VR teleoperation using a Meta Quest Pro headset and handheld controllers. The controllers command arm motion and gripper opening and closing.

Bimanual Training. We train on 90 demonstrations for `Reactive Grasp` and 19 successful demonstrations for `Tomato Plate Transfer`; each task is trained separately on all of its demonstrations with no held-out episodes, and Standard BC and Anchor-Align are trained on the same demonstrations. Both are trained with VLA-Adapter for 10K steps with identical hyperparameters.

Bimanual direction labels. YAM has two arms, so its 14-dimensional action concatenates two 7-DoF arm actions, each with a translational component $(\Delta x, \Delta y, \Delta z)$, a rotational component, and a gripper command. The two arm bases are parallel, so we simply duplicate the label construction of Sec. 3.3: the dominant-axis rule is applied to each arm's $(\Delta x, \Delta y, \Delta z)$ separately, with the same axis-to-word mapping for both arms. Each arm thus receives its own direction word (e.g., *up* for both arms while lifting the plate), and the alignment loss is averaged over the two arms.

B.5 Alignment vs. Regularization: Shuffle and Scatter Controls

This section gives the full construction of the *Shuffle* and *Scatter* controls; their motivation and headline results are covered in Sec. 4.3 and Tab. 4. Both controls are trained with hyperparameters identical to Anchor-Align VLA: the only quantity that changes is the lookup table from motion class to target token id, so here we are destroying semantic content in these controls.

***Shuffle*.** The observation-to-label mapping is reshuffled once and held fixed throughout training. Concretely, we apply a fixed derangement of the six direction words: *forward* $\rightarrow$ `right`, *backward* $\rightarrow$ `down`, *left* $\rightarrow$ `forward`, *right* $\rightarrow$ `up`, *up* $\rightarrow$ `left`, and *down* $\rightarrow$ `backward`.

***Scatter*.** Each direction class is mapped one-to-one to a fixed meaningless word: *forward* $\rightarrow$ `purple`, *backward* $\rightarrow$ `seven`, *left* $\rightarrow$ `today`, *right* $\rightarrow$ `music`, *up* $\rightarrow$ `tree`, and *down* $\rightarrow$ `math`. The six words are chosen to be mutually distant in the frozen output-embedding space, and the targets remain perfectly synchronized with the observations, so the model still solves a well-defined six-way classification task; only the motion semantics of the target tokens are removed.

Per axis, the collapse reported in Tab. 4 is sharpest on the LIBERO-PRO position-swap axis, where success requires re-grounding language to the perturbed scene rather than memorizing scene-to-action mappings: Anchor-Align VLA achieves 22.6% on position swap while *Shuffle* and *Scatter* recover only the baseline's near-zero performance.

C Extended Quantitative Results

This section complements the main paper's quantitative evaluation (Sec. 4.2) with benchmark subcategory details, success rates on the standard LIBERO suites, per-suite robustness breakdowns, multi-seed significance tests, and backbone regularization controls.

C.1 LIBERO-PRO and LIBERO-Plus Subcategory Clarifications

LIBERO-PRO language rephrase rewords the instruction ("pick up the mug" becomes "grab the mug"), whereas LIBERO-Plus language instruction rewrites it globally, e.g., referring to the target by its function ("the flat surface used for holding food" for a plate), forcing the policy to ground the words in the observed scene. Similarly, LIBERO-PRO object swap replaces scene objects with unseen variants and position swap exchanges their locations (Fig. 4), whereas LIBERO-Plus object layout keeps the objects but shifts the target location and adds distractors.

Table 6: **Success rates across standard LIBERO suites.** Anchoring and alignment achieve state-of-the-art results on unperturbed (standard) benchmarks. Best per column is **bolded**.

Method	Spatial	Object	Goal	Long
OpenVLA-OFT (Kim et al., 2025)	94.3	95.2	91.7	86.5
MolmoAct (Lee et al., 2025)	87.0	95.4	87.6	77.2
$\pi_{0.5}$-KI (Driess et al., 2025)	96.6	97.2	94.6	85.8
VLA-0 (Goyal et al., 2025)	93.6	96.0	95.6	87.6
VLA-Adapter[Frozen] (Wang et al., 2025)	89.4	89.6	88.0	84.5
VLA-Adapter (Wang et al., 2025)	96.0	**99.8**	96.0	89.0
Anchor-Align VLA	**98.4**	**99.8**	**97.2**	**90.8**

C.2 Success Rates on Standard LIBERO Suites

We report success rates on the four standard (unperturbed) LIBERO suites—Spatial, Object, Goal, and Long—in Tab. 6. Anchor-Align VLA achieves the highest success rate on most suites, surpassing prior methods including $\pi_{0.5}$-KI and OpenVLA-OFT, which use substantially larger backbones and large-scale robotic pretraining. This confirms that the gains from anchoring and alignment hold not only on the OOD stress tests reported in the main paper (Tab. 1), but also on the in-distribution benchmark.

C.3 Per-Suite Robustness Breakdowns

In the main paper (Tab. 1), we report robustness and generalization results on the LIBERO-Spatial suite, where perturbations to object positions, language instructions, and scene layout directly test whether the policy has learned transferable spatial reasoning or merely memorized training trajectories. Here we show that the same gains carry over to the remaining three LIBERO suites: Long (Fig. 13), Object (Fig. 14), and Goal (Fig. 15), which test long-horizon composition, object knowledge, and procedural reasoning, respectively. All comparisons are against VLA-Adapter, the strongest standard BC baseline from Tab. 1. Because VLA-Adapter uses the same LoRA architecture and training data as Anchor-Align VLA but without anchoring or alignment, it directly isolates the effect of our two proposed objectives. Each radar plot reports performance across nine evaluation axes: two from LIBERO-PRO (Language Rephrase and Object Swap) and seven from LIBERO-Plus (Language Instruction, Background Texture, Robot Init State, Camera Viewpoint, Object Layout, Lighting Condition, and Sensor Noise). Standard and Position Swap axes are excluded.

C.4 Multi-Seed Evaluation: Statistical Significance

To confirm that the gains reported in the main paper's Tab. 1 (LIBERO-PRO and LIBERO-Plus) are not artifacts of a single random seed, we re-run both Anchor-Align VLA and the VLA-Adapter baseline with five independent training seeds and report mean $\pm$ standard deviation in Tab. 7. The standard deviations are small (all ≤ 0.7) and the mean improvements of Anchor-Align VLA over VLA-Adapter on every column are substantially larger than the combined seed-to-seed variability, so the differences are statistically significant by a paired comparison. The multi-seed means also closely match the single-seed numbers reported in Tab. 1, indicating that the headline results are stable rather than seed-lucky.

Table 7: **Multi-seed evaluation (5 seeds).** Mean $\pm$ standard deviation of success rate (%) for Anchor-Align VLA and the VLA-Adapter baseline on LIBERO-PRO language rephrase / object swap / position swap, and the LIBERO-Plus overall average. Seed-to-seed variability is small (≤ 0.7) and substantially smaller than the gap between methods, confirming that the improvements reported in the main paper's Tab. 1 are statistically significant.

Method	Lang. Reph.	Object Swap	Pos. Swap	Plus
VLA-Adapter	91.1 ± 0.4	90.1 ± 0.5	2.6 ± 0.7	85.3 ± 0.3
Anchor-Align VLA	$\mathbf{97.1 \pm 0.5}$	$\mathbf{96.1 \pm 0.4}$	$\mathbf{23.5 \pm 0.2}$	$\mathbf{90.5 \pm 0.6}$

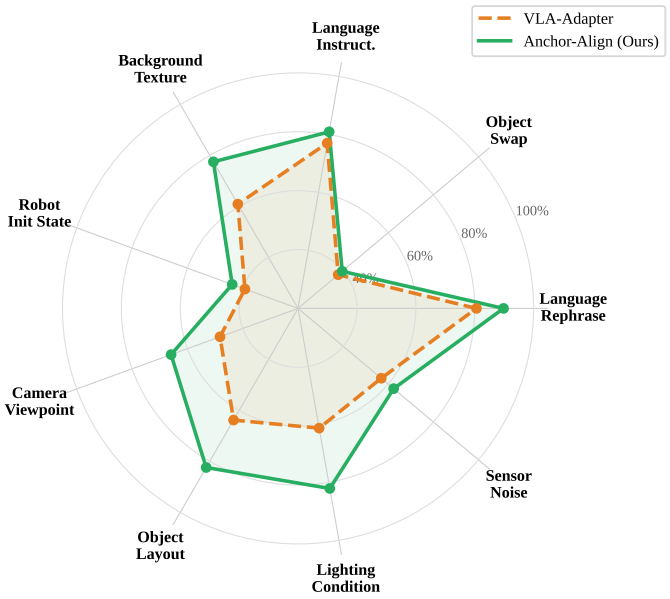


Figure 13: **LIBERO Long Suite.** Performance across nine evaluation axes (excluding Standard and Position Swap). Anchor-Align VLA (green) substantially expands coverage over VLA-Adapter (orange), with the largest gains on Lighting Condition (+20.8%), Object Layout (+18.6%), and Camera Viewpoint (+17.7%).

Gains transfer to a larger VLM backbone. To verify that the improvements from Anchor-Align are not specific to our default backbone or action head, we re-instantiate the method on a different StarVLA configuration than the one used in the main-paper real-world experiments (Sec. 4.4). Recall that StarVLA is a modular VLA framework that combines an arbitrary VLM backbone with an arbitrary action head; here we instantiate it with a larger Qwen2.5 3B VLM and an OFT MLP-based action head, in contrast to the Qwen2.5-VL + GR00T FM-DiT instantiation used in the main paper. On LIBERO-PRO, Anchor-Align VLA improves the StarVLA baseline from 73.6% to $\mathbf{89.0}\%$ on the language rephrase axis and from 89.8% to $\mathbf{91.2}\%$ on the object swap axis. The improvement is qualitatively consistent with what we observe on the smaller backbone, indicating that the benefits of anchoring and alignment are a property of the training objective rather than of a particular VLM scale or action-head design.

C.5 Backbone Regularization Controls

We compare Vision-Language Anchoring against five alternative backbone regularization settings: reducing the LoRA rank to $r = 32$, reducing the learning rate by a factor of ten, post-hoc interpolation between the adapted and frozen backbone weights at $\alpha = 0.9$ and $\alpha = 0.75$, and applying adapter weight decay of 0.1. As shown in Tab. 8, anchoring alone achieves the highest mean success rate on both LIBERO-PRO and LIBERO-Plus, exceeding the strongest alternative, adapter weight decay, by 5.7 and 1.7 percentage points, respectively. These results support explicitly preserving pretrained representations through anchoring over the tested controls that limit adaptation through capacity, optimization, or weight-space regularization.

D Representation Analysis: Language Preservation and Action Decodability

This appendix provides the full mathematical details for the evaluation metrics used in the paper, followed by the representation-level analyses built on them.



Figure 14: **LIBERO Object Suite.** Performance across nine evaluation axes (excluding Standard and Position Swap). Anchor-Align VLA (green) consistently covers a larger area than VLA-Adapter (orange), with the most pronounced gains on Robot Init State (+18.6%).

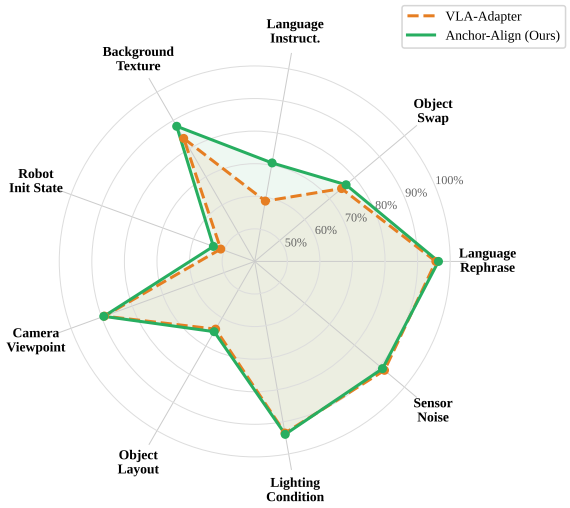


Figure 15: **LIBERO Goal Suite.** Performance across nine evaluation axes (excluding Standard and Position Swap). Anchor-Align VLA (green) shows the largest gains over VLA-Adapter (orange) on Language Instruction (+11.9%) and Background Texture (+4.3%).

D.1 Centered Kernel Alignment (CKA)

We use CKA (Kornblith et al., 2019; Bo et al., 2024) to measure how much of the pretrained VLM's representational geometry is preserved after finetuning. CKA is built on the Hilbert–Schmidt Independence Criterion (HSIC) (Gretton et al., 2005), which quantifies the statistical dependence between two sets of representations via their kernel matrices.

HSIC. Given n data points and two representation matrices $\mathbf{X} \in \mathbb{R}^{n \times p}$ and $\mathbf{Y} \in \mathbb{R}^{n \times q}$, we form the Gram matrices $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$ and $\mathbf{L} = \mathbf{Y}\mathbf{Y}^\top$ (under a linear kernel). The empirical HSIC is

$$\mathrm{HSIC}(\mathbf{K}, \mathbf{L}) = \frac{1}{(n-1)^2} \operatorname{tr}\big(\mathbf{K}\,\mathbf{H}\,\mathbf{L}\,\mathbf{H}\big), \tag{4}$$

where $\mathbf{H} = \mathbf{I}_n - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$ is the centering matrix. Centering removes the mean from each kernel matrix, so that HSIC captures the structural (second-order) similarity rather than mere magnitude.

Table 8: **Backbone regularization comparison.** Mean success rates (%) on LIBERO-PRO (PRO) and LIBERO-Plus (Plus). Adapter↔frozen denotes post-hoc adapter scaling, equivalent to weight-space interpolation with the frozen backbone. Anchoring alone outperforms all five alternative settings on both benchmarks.

Backbone regularization	**PRO**	**Plus**
Reduced LoRA rank ($r = 32$)	60.0	80.4
Reduced learning rate ($\times 0.1$)	55.8	71.3
Adapter↔frozen ($\alpha = 0.9$)	61.8	85.0
Adapter↔frozen ($\alpha = 0.75$)	59.0	82.5
Adapter weight decay (0.1)	62.4	85.6
Anchor VLA (anchoring only)	**68.1**	**87.3**

CKA. CKA normalizes HSIC so that the score lies in $[0, 1]$:

$$\mathrm{CKA}(\mathbf{K}, \mathbf{L}) = \frac{\mathrm{HSIC}(\mathbf{K}, \mathbf{L})}{\sqrt{\mathrm{HSIC}(\mathbf{K}, \mathbf{K})\ \mathrm{HSIC}(\mathbf{L}, \mathbf{L})}}. \tag{5}$$

A CKA score of 1.0 indicates that the two representations have identical geometric structure up to an isotropic scaling and orthogonal transformation; lower values indicate that finetuning has reshaped the layer's representations.

Linear CKA. When the linear kernel is used, the CKA expression simplifies to a closed-form that avoids explicitly constructing the $n \times n$ Gram matrices. Let $\tilde{\mathbf{X}}$ and $\tilde{\mathbf{Y}}$ denote the column-centered versions of $\mathbf{X}$ and $\mathbf{Y}$. Then:

$$\mathrm{CKA}_{\text{linear}}(\mathbf{X}, \mathbf{Y}) = \frac{\left\|\tilde{\mathbf{Y}}^\top \tilde{\mathbf{X}}\right\|_F^2}{\left\|\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}\right\|_F \cdot \left\|\tilde{\mathbf{Y}}^\top \tilde{\mathbf{Y}}\right\|_F}. \tag{6}$$

This is the form we use throughout: at each transformer layer u, $\mathbf{X} \in \mathbb{R}^{n \times d}$ contains the text-token hidden states from the frozen pretrained VLM and $\mathbf{Y} \in \mathbb{R}^{n \times d}$ from the finetuned backbone VLM, across n evaluation samples. We report text-token CKA because it directly reflects the degree to which the model's language understanding is preserved.

D.2 Action Decodability (Linear Probing R^2)

To measure how much action-relevant information is linearly accessible from the model's hidden states, we train a ridge regression probe at each layer. For layer u, we extract the hidden state and mean-pool over all token positions to obtain a feature vector $\mathbf{f}_i^{(u)} \in \mathbb{R}^d$ for each sample i. We then fit a ridge regression model g to predict the ground-truth discretized action $\mathbf{a}_i \in \mathbb{R}^7$ (with 256 bins per dimension):

$$R^2(u) = 1 - \frac{\sum_{i=1}^n \left\|\mathbf{a}_i - g\big(\mathbf{f}_i^{(u)}\big)\right\|_2^2}{\sum_{i=1}^n \left\|\mathbf{a}_i - \bar{\mathbf{a}}\right\|_2^2}, \tag{7}$$

where $\bar{\mathbf{a}}$ is the mean action over the dataset. A higher R^2 indicates that more action information is linearly decodable from that layer's representations, even without the action head.

D.3 Language Preservation and Action Decodability

We quantify what each training paradigm does to the backbone's internal representations along two axes: language preservation, measured with Centered Kernel Alignment (CKA) (Bo et al., 2024) between the finetuned backbone's text-token hidden states and the pretrained VLM's (formal definition in App. D.1), and action decodability, measured by linear-probing the backbone's hidden states for the ground-truth action (details in App. D.2). As shown in Fig. 16, standard BC catastrophically destroys the pretrained text representations (CKA drops to 0.34; per-layer analysis in App. D.4). At the opposite extreme, the frozen backbone trivially preserves the pretrained geometry yet gains no action information, and the co-trained VLA behaves similarly. Anchor-Align VLA achieves the

best of both: it maintains strong language preservation (CKA = 0.91) while attaining the highest action decodability of any method (peak R^2 = 0.60 at layer 22), showing that alignment routes action-relevant information through the decoder layers without overwriting the pretrained language geometry. Preservation captures what the model keeps from pretraining, decodability what it gains from finetuning; together they show that Anchor-Align VLA attains high task performance by enriching the pretrained representations rather than destroying them, as standard BC does. Additional linear-probe results on direction concepts (in-distribution vs. OOD) are reported in App. D.5.

D.4 Layer-wise Language Preservation

App. D.3 reports language preservation per model; here we break the same measurement down by decoder layer, asking whether the GQA collapse in Fig. 5 is only behavioral or whether finetuning actually reshapes the backbone's internal representations. Fig. 17 plots the text-token CKA between the finetuned backbone and the pretrained VLM at every layer (formal definition in App. D.1). Standard BC progressively destroys the pretrained text representations with depth, collapsing in the output layers (CKA drops to 0.34 at layer 24), the same layers that drive the GQA collapse, so the degradation is representational rather than merely behavioral. Anchor VLA instead recovers near-perfect preservation through layer-wise distillation, with an average CKA of 0.95 across layers.

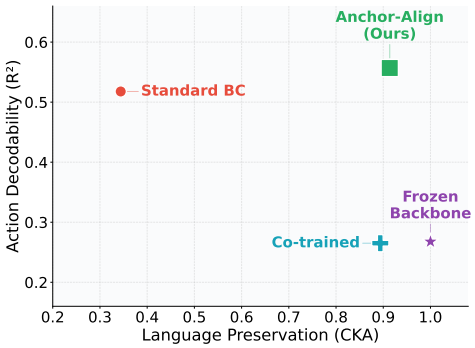


Figure 16: **Action decodability vs. language preservation.** Per-model language preservation (text-token CKA) vs. action decodability (linear-probe R^2): Anchor-Align VLA attains the highest action decodability while sustaining high language preservation, gaining action information without overwriting the pretrained geometry.

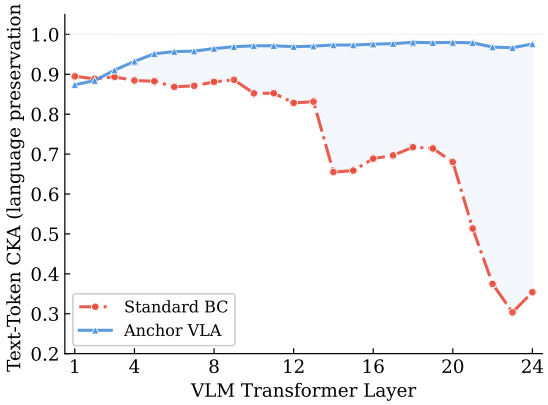


Figure 17: **Layer-wise language preservation.** Per-layer text-token CKA between the finetuned backbone and the pretrained VLM. Standard BC collapses the pretrained text representations in the output layers, whereas Anchor VLA preserves them across all layers through layer-wise distillation.

D.5 Directional Information in Backbone Representation

To further unpack what Anchor-Align changes inside the backbone, we linearly probe six motion-direction classes from the final-layer hidden state immediately before the action tokens. We compare the frozen backbone, VLA-Adapter, Anchor-only VLA, and Anchor-Align VLA using the same probing protocol: a separate linear classifier is fitted to each fixed backbone on 8,000 Spatial samples for five epochs and evaluated on 1,500 samples each from Spatial (in-distribution) and Object (out-of-distribution). Only the linear classifier is optimized during probing. In every case the probe reads the pre-action hidden state $\mathbf{h}^{\text{pre}}$ (Sec. 3.3) directly from the backbone. For Anchor-Align VLA, this bypasses the learned projection $\mathbf{W}_{\text{proj}}$ and the frozen language head used by the alignment loss, so the accuracies reflect directional information encoded in the VLM backbone itself rather than in the alignment head.

Tab. 9 shows that Anchor-only VLA improves over the frozen backbone by 26.9 percentage points on Spatial and 5.7 points on Object. Relative to VLA-Adapter, anchoring alone yields lower Spatial accuracy but higher Object accuracy (28.5% vs. 21.4%), consistent with preserving more transferable directional information. Anchor-Align VLA achieves the highest accuracy on both suites.

Together, these results show that Anchor-Align VLA makes motion-direction information substantially more accessible in the VLM backbone, with the improvement over Anchor-only isolating the contribution of Language-Action Alignment (see Tab. 10).

Table 9: **Direction understanding.** Six-way linear-probe accuracy (%) from fixed backbone representations. Anchor-only VLA improves over the frozen backbone on both suites and over VLA-Adapter on Object OOD; Anchor-Align VLA achieves the highest reported accuracy on both suites.

Method	**Spatial (in-dist.)**	**Object (OOD)**
Frozen VLA-Adapter backbone	43.8	22.8
VLA-Adapter	86.0	21.4
Anchor-only VLA	70.7	28.5
Anchor-Align VLA	**94.4**	**30.3**

E Language-Action Alignment Diagnostic

E.1 Diagnostic Protocol

Sec. 3.3 defines the training-time direction labels and alignment loss, and App. B.1 specifies the alignment head. Here we describe the evaluation protocol for measuring whether the language and action heads agree on motion direction during robot rollouts.

Per-frame action labels. For a trajectory frame t, we extract the world-coordinate translational displacement $\mathbf{d}_t = (\Delta x_t, \Delta y_t, \Delta z_t)$ from the policy action and apply the dominant-axis rule in Sec. 3.3. The axis pairs map to forward/backward (x), left/right (y), and up/down (z), with ties resolved in the order x, y, z. This produces an action-direction label ℓ_t^{act} for each frame.

Diagnostic frame selection. We retain a frame only if $\|\mathbf{d}_t\|_2 \geq \mu$, with $\mu = 0.001$, and its direction label agrees with those of the next five frames: $\ell_t^{\text{act}} = \ell_{t+j}^{\text{act}}$ for $j \in \{1, \ldots, 5\}$. These criteria remove near-stationary and transient motions from the diagnostic. They are distinct from the training-time filter on chunk-averaged translations, whose threshold τ is specified in App. B.1.

Language-action agreement. Given VLMs' ability to reason about spatial relations in visual question answering (Chen et al., 2024; Song et al., 2025; Dalal et al., 2026a;c), we test whether the backbone VLM's answers to motion-direction questions agree with the finetuned VLA's predicted actions. On the same evaluated observation, we query the language head with a templated question about the robot's immediate next motion: *"Q: In which immediate direction should the end effector move to place the bowl on the plate? Options: (a) Left, (b) Right, (c) Up, (d) Down, (e) Forward, (f) Backward."* The language prediction is interpreted in the camera-centric coordinate frame, i.e., left/right, up/down, and forward/backward denote the corresponding directions relative to the camera view. Because the policy action is expressed in the world/action coordinate frame, we convert the predicted camera-frame direction to the world frame using the known camera-to-world coordinate mapping. We then apply the same dominant-axis/sign convention used for the action displacement, yielding a world-frame language-direction label ℓ_t^{lang}. Language-action agreement is measured as $\mathbb{1}[\ell_t^{\text{lang}} = \ell_t^{\text{act}}]$. This evaluation query is separate from the training loss, which supervises direction logits from the pre-action hidden state (Sec. 3.3). Alignment is the fraction of evaluated observations for which the converted language prediction agrees with the action-direction label, regardless of whether the shared prediction is correct. We additionally compute alignment per rollout and correlate it with binary task success using Pearson r.

E.2 Relationship to Task Success

Table 10: **Language-action alignment on LIBERO-PRO rollouts.** Success and alignment are percentages; Pearson r correlates per-rollout alignment with task success.

VLA Model	Success	Alignment	Pearson r
Co-training + KI (Driess et al., 2025)	48.5	14.3	$+0.10$
VLA-Adapter (Wang et al., 2025)	61.0	16.8	-0.03
MolmoAct (Lee et al., 2025)	53.5	13.2	-0.18
Anchor-Align VLA	**71.9**	**78.4**	$\mathbf{+0.51}$

Tab. 10 applies this diagnostic to LIBERO-PRO rollouts and reports task success alongside the agreement metrics defined above. Co-training + KI is our VLA-Adapter-based reimplementation described in App. A.

VLA-Adapter and Co-training + KI show low alignment (16.8% and 14.3%) and weak correlations with task success ($r = -0.03$ and $+0.10$). MolmoAct likewise has low alignment (13.2%), with a negative correlation ($r = -0.18$). Anchor-Align VLA raises alignment to 78.4% and yields a positive correlation of $r = +0.51$, alongside 71.9% task success compared with 61.0% for VLA-Adapter. Thus, higher agreement is associated with successful rollouts for Anchor-Align VLA, while agreement alone is not a reliable success indicator across all evaluated methods. This correlation does not establish a causal effect of alignment on task success.

F Real-World Rollouts

F.1 Bimanual Height Coordination

We compare vertical coordination on Tomato Plate Transfer between Standard BC and Anchor-Align using the same dual-arm YAM Ultra setup, parallel arm bases, and plate placement (App. B.4). Gripper heights are reconstructed from recorded joint states using forward kinematics and smoothed with a centered nine-frame cubic filter before extracting the analysis interval.

We analyze 20 trials per method over the first 30 samples at 30 Hz after t_0 (0–0.967 s), where t_0 is the later of the two grippers' first sustained closures from an open state (below 0.2 for three consecutive samples).

Let $\Delta z_a(t) = z_a(t) - z_a(t_0)$ be the height change of arm $a \in \{L, R\}$ from closure. We measure each trial's root-mean-square (RMS) height-change mismatch as

$$E_z = \sqrt{\frac{1}{N}\sum_{k=0}^{N-1}\big[\Delta z_R(t_k) - \Delta z_L(t_k)\big]^2}, \qquad N = 30, \quad t_k = t_0 + \frac{k}{30}\,\text{s}. \tag{8}$$

The mean of E_z across trials decreases from 1.69 mm for Standard BC to 0.91 mm for Anchor-Align, a 46.2% reduction (Fig. 18). The lower mismatch provides kinematic evidence of more coordinated vertical motion during the initial post-closure interval. This metric measures changes in relative gripper height; it does not directly establish plate orientation or isolate the contribution of either training loss.

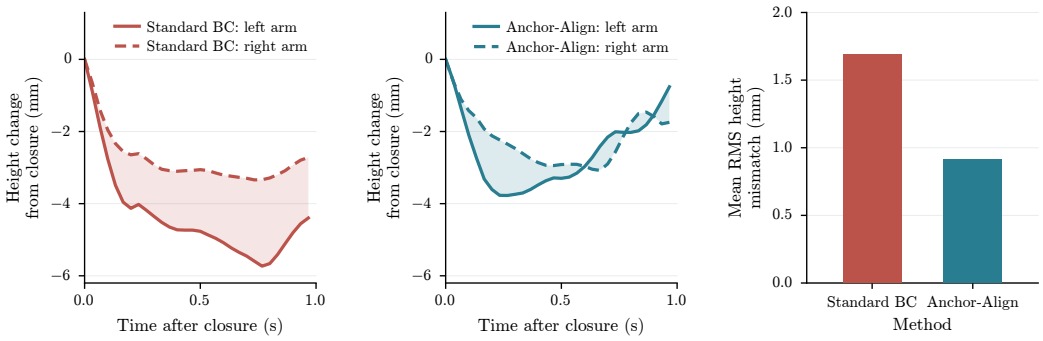


Figure 18: **Bimanual height coordination after gripper closure.** Left and middle: example Standard BC (red) and Anchor-Align (teal) trajectories; solid and dashed curves show the left and right grippers' height changes from closure, with shading marking their difference. Right: mean per-trial RMS height-change mismatch across all eligible trials, showing lower mismatch with Anchor-Align.

F.2 Object-Orientation Perturbation

Beyond swapping object positions, our Broccoli Placement task also perturbs the *orientation* of the target object itself. Fig. 19 shows the broccoli used in the pick-and-place task in six distinct poses: varying the object's orientation while holding its identity and the instruction fixed prevents the policy from matching a memorized canonical view and forces it to re-ground perception in each rollout.

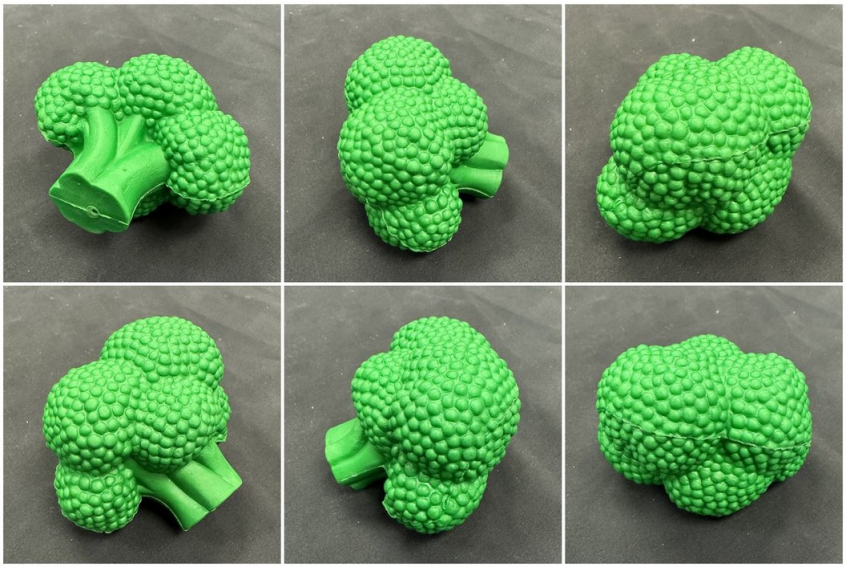


Figure 19: **Target-object orientation perturbation in Broccoli Placement.** The same broccoli, cropped from the front-view camera, shown in six distinct orientations used across evaluation rollouts of the "Pick up the green broccoli and place it on the plate task". We perturb the orientation of the object *itself*, not just its position, so its silhouette and visible structure change substantially while its identity and the language instruction remain fixed. This prevents the policy from relying on a canonical, memorized view of the target and, combined with the position and distractor swaps in Fig. 20, makes the task considerably harder than the position-only swap of LIBERO-PRO.

F.3 ANCHOR-ALIGN REAL-WORLD ROLLOUTS

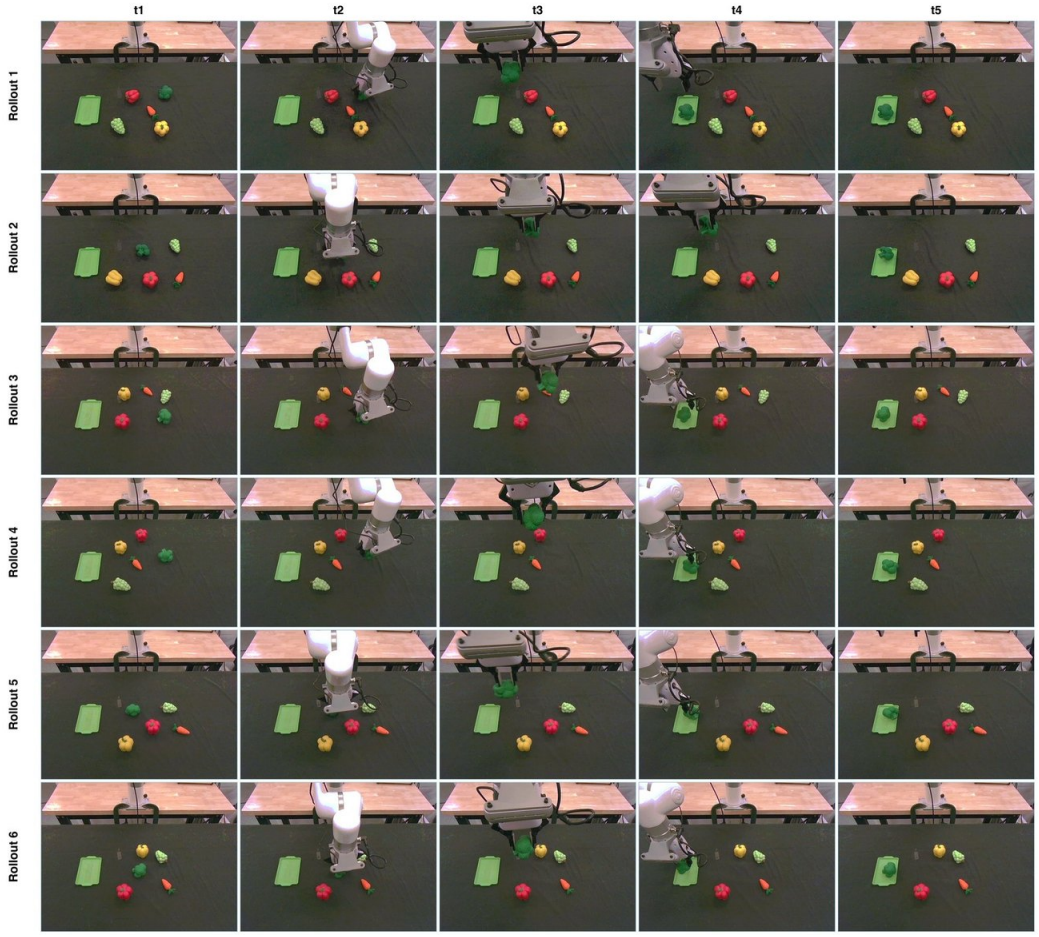


Figure 20: **Broccoli Placement rollouts from the Anchor-Align VLA model.** The task is to pick up the broccoli and place it on the green plate. **The broccoli's spatial position and the surrounding distractor objects are simultaneously swapped across rollouts**, so each rollout is evaluated on a unique scene configuration. All six rollouts succeed, showing that Anchor-Align VLA handles joint object-swap and position-swap perturbations robustly without relying on memorized scene configurations. Each row shows 5 keyframes from a single rollout.

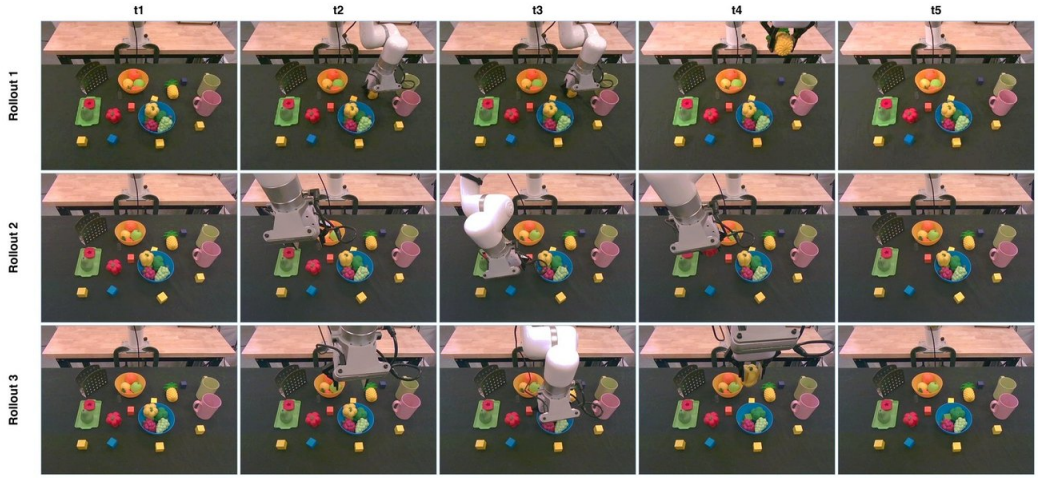


Figure 21: **Real-world rollouts from the Anchor-Align VLA model.** The task is to pick up the object named by the language instruction (pineapple, red bell pepper, or yellow bell pepper) and place it on the plate in a **very cluttered scene**. **The target is specified by text while many distractor objects (including the other candidate items) crowd the workspace**, so the policy must ground the instruction in the current observation and grasp the correct object rather than defaulting to a memorized choice. All three rollouts succeed, showing that Anchor-Align VLA discriminates the language-specified target under heavy clutter and visually similar distractors. Rollouts 1, 2, and 3 target the pineapple, the red bell pepper, and the yellow bell pepper, respectively. Each row shows 5 keyframes from a single rollout.

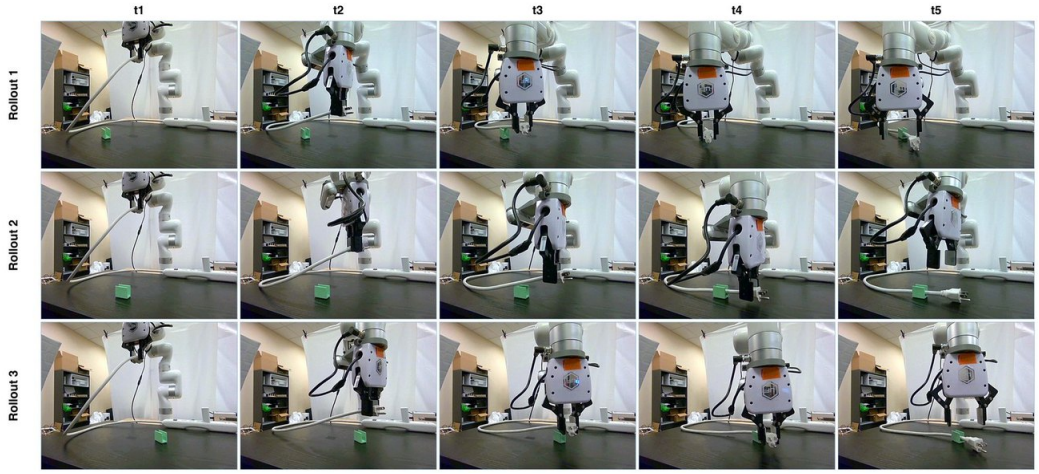


Figure 22: **Cable Insertion rollouts from the Anchor-Align VLA model.** The task is to insert the wire into the green cable manager, an OOD target that replaces the red cable manager used during training. **Each row is a successful rollout in a different spatial configuration of the cable manager.** Each row shows 5 evenly spaced keyframes from a single rollout.

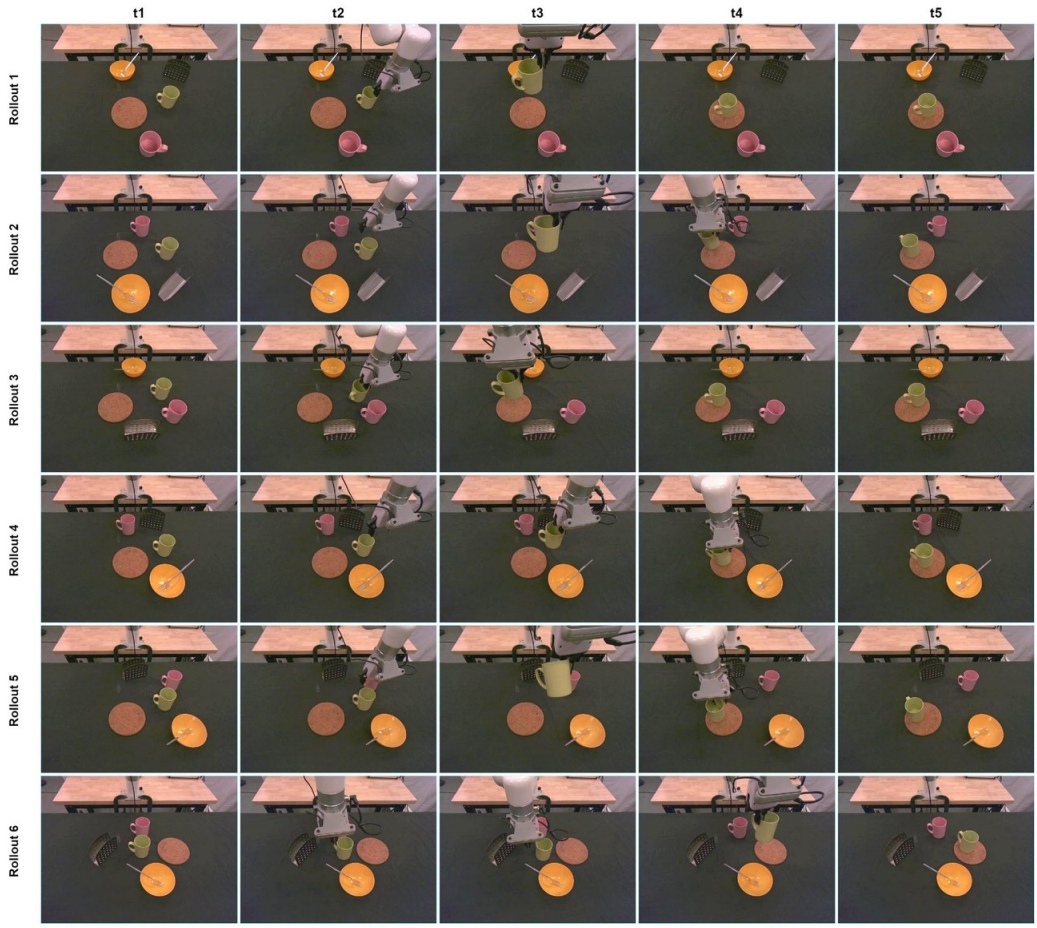


Figure 23: **Green Mug Placement rollouts from the Anchor-Align VLA model.** The task is to pick up the green mug and place it on the plate. **The mug's spatial position and the surrounding distractor objects are simultaneously swapped across rollouts**, so each rollout is evaluated on a unique scene configuration. All six rollouts succeed, showing that Anchor-Align VLA handles joint object-swap and position-swap perturbations robustly without relying on memorized scene configurations. Each row shows 5 evenly spaced keyframes from a single rollout.

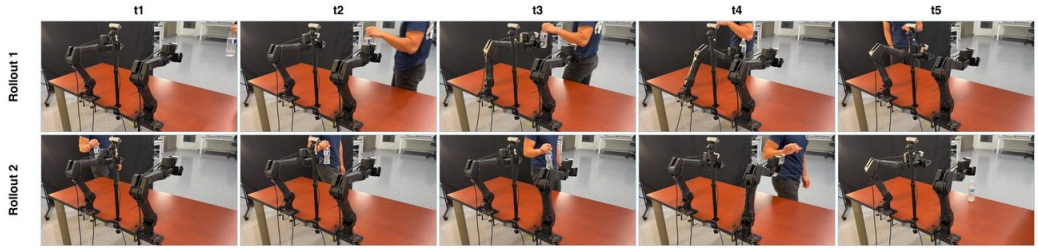


Figure 24: **Reactive Grasp rollouts from the Anchor-Align VLA model.** A person carries a bottle across the robot's workspace, and the robot grasps the bottle while it moves. The person walks in opposite directions in the two rollouts, so a different arm makes the grasp. Each row shows 5 keyframes from a single rollout.

G QUALITATIVE RESULTS IN LIBERO SIMULATOR

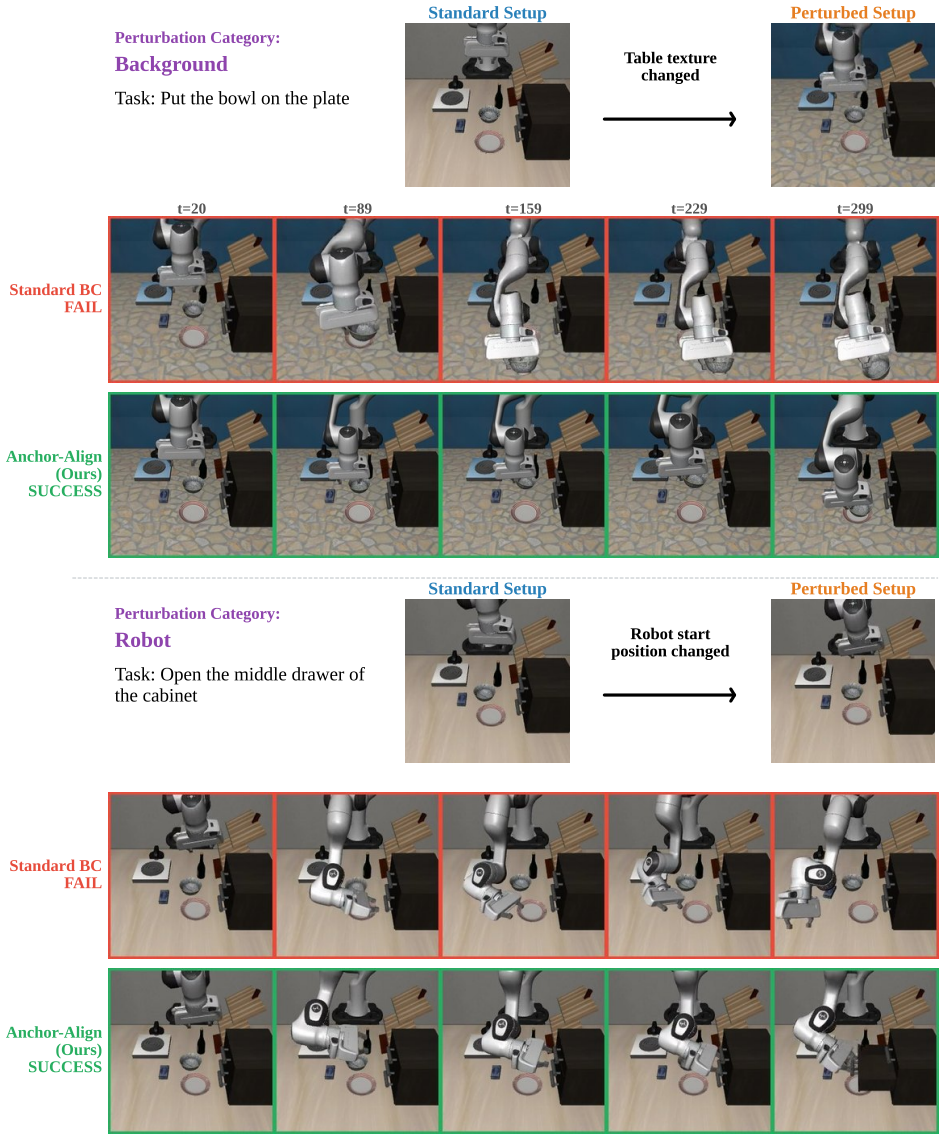


Figure 25: **LIBERO-Goal Plus: *Background* and *Robot* perturbations.** *Top:* Background perturbation on "Put the bowl on the plate"—the table texture is changed to stone tile. Standard BC (red) collapses onto the table without releasing the bowl on the plate; Anchor-Align VLA (green) places the bowl on the plate. *Bottom:* Robot perturbation on "Open the middle drawer of the cabinet"—the robot's initial joint configuration is shifted. Standard BC approaches the wrong side of the cabinet and never opens the drawer; Anchor-Align VLA locates the middle drawer and pulls it open.

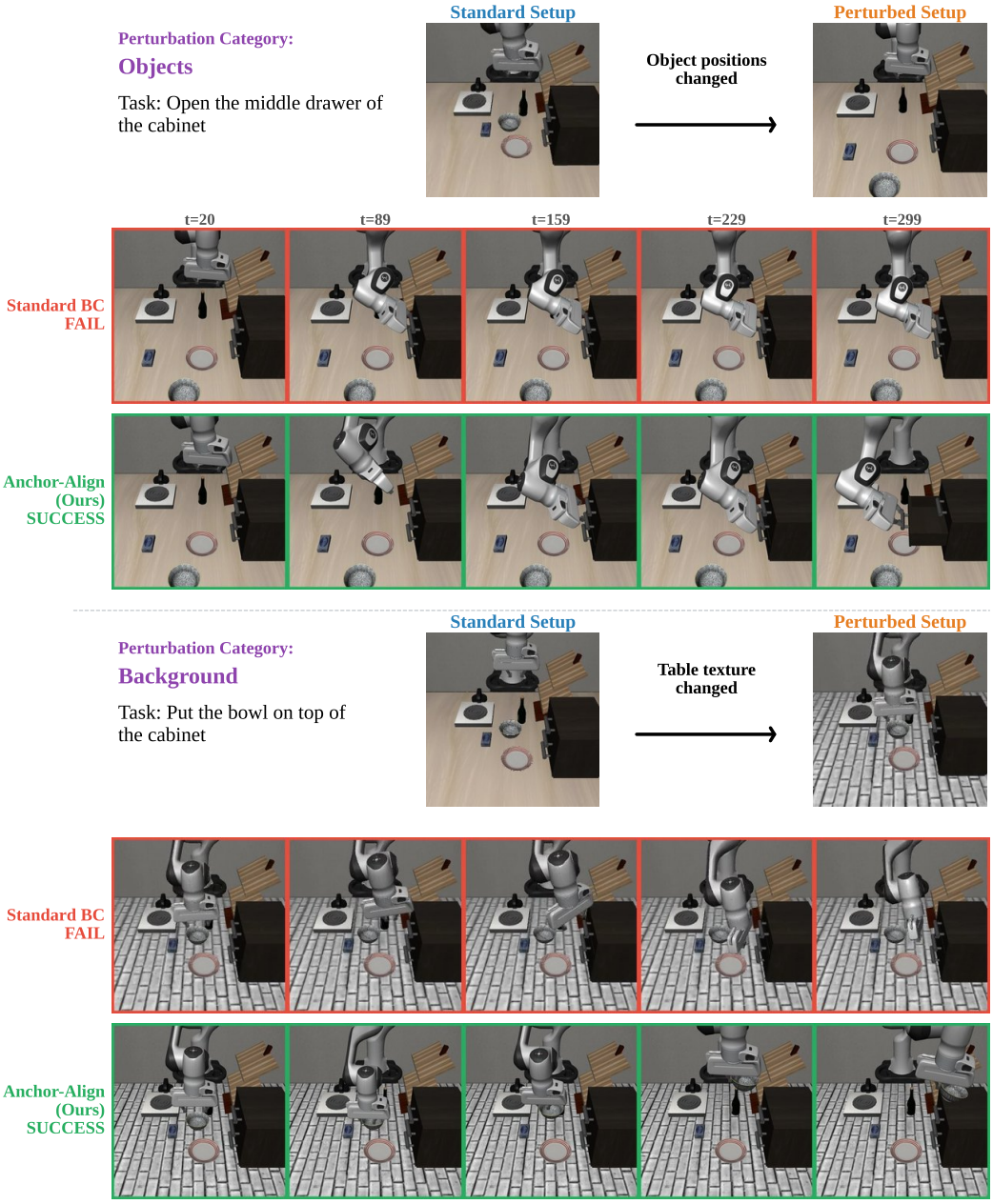


Figure 26: **LIBERO-Goal Plus: *Objects* and *Background* perturbations.** *Top:* Objects perturbation on "Open the middle drawer of the cabinet"—the cabinet, plate, and surrounding distractors are rearranged. Standard BC (red) hovers above the wrong region without contacting the handle; Anchor-Align VLA (green) locates the middle drawer and opens it. *Bottom:* Background perturbation on "Put the bowl on top of the cabinet"—the wood floor is swapped for a stone-tile surface. Standard BC drifts away from the cabinet; Anchor-Align VLA grasps the bowl and deposits it on top of the cabinet.

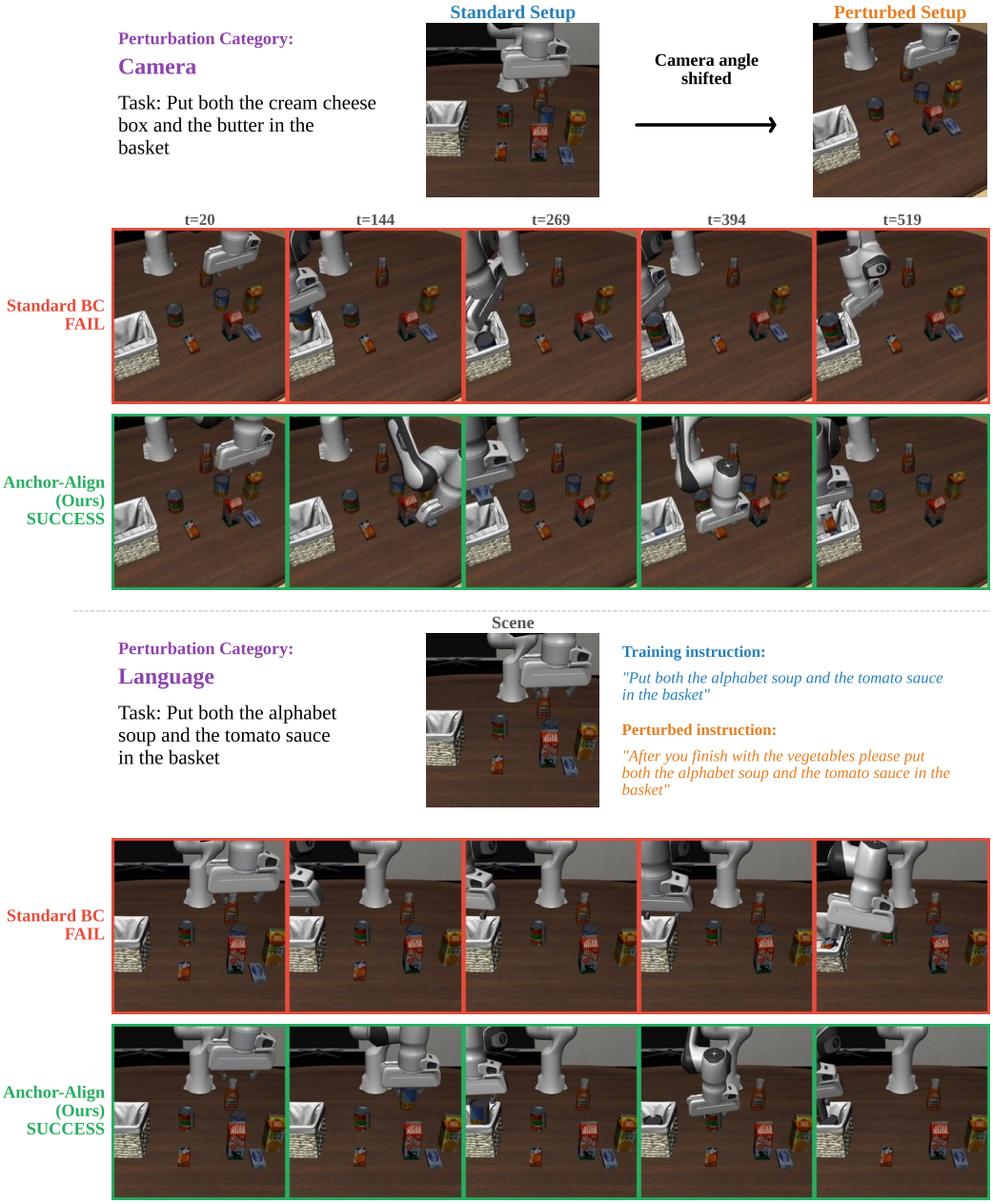


Figure 27: **LIBERO-Long Plus: *Camera* and *Language* perturbations.** *Top:* Camera perturbation on the long-horizon task "Put both the cream cheese box and the butter in the basket": the camera viewpoint is shifted. Standard BC (red) picks up the incorrect object and places it in the basket, showcasing its reliance on memorized scene-to-action mappings that break under the viewpoint shift; Anchor-Align VLA (green) sequentially picks both targets and drops them in the basket. *Bottom:* Language perturbation on "Put both the alphabet soup and the tomato sauce in the basket": the instruction is rephrased to "*After you finish with the vegetables please put both the alphabet soup and the tomato sauce in the basket*". Standard BC fails to initiate the correct grasps; Anchor-Align VLA follows the rephrased instruction to completion.

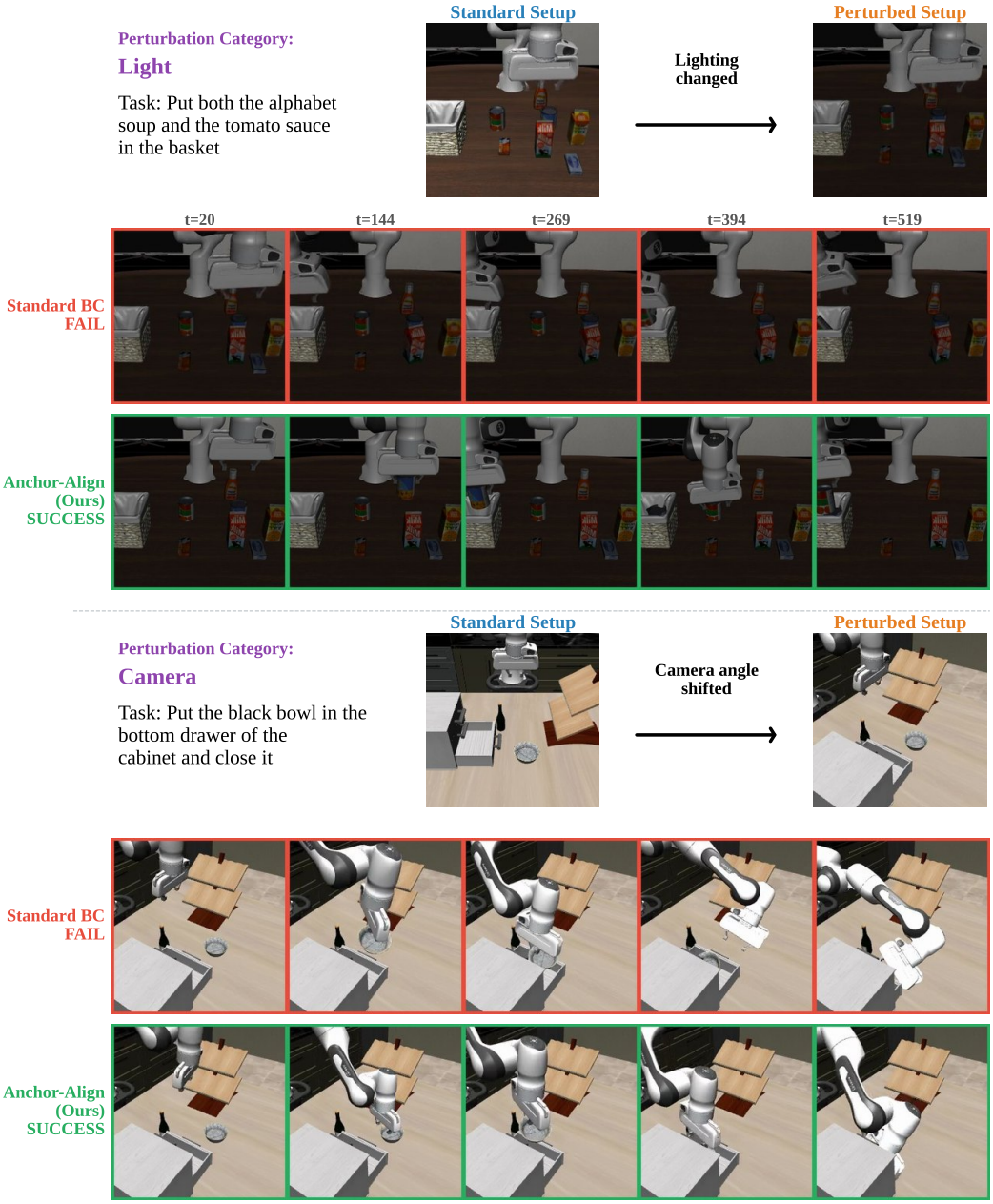


Figure 28: **LIBERO-Long Plus: *Lighting* and *Camera* perturbations.** *Top:* Lighting perturbation on "Put both the alphabet soup and the tomato sauce in the basket": the scene is darkened. Standard BC (red) grasps an incorrect object, places it in the basket, and then idles without ever reaching for the second target; Anchor-Align VLA (green) deposits both items in the basket. *Bottom:* Camera perturbation on the long-horizon task "Put the black bowl in the bottom drawer of the cabinet and close it": the camera viewpoint is shifted. Standard BC pulls the wrong drawer and never completes the close step; Anchor-Align VLA opens the bottom drawer, places the black bowl inside, and closes it.

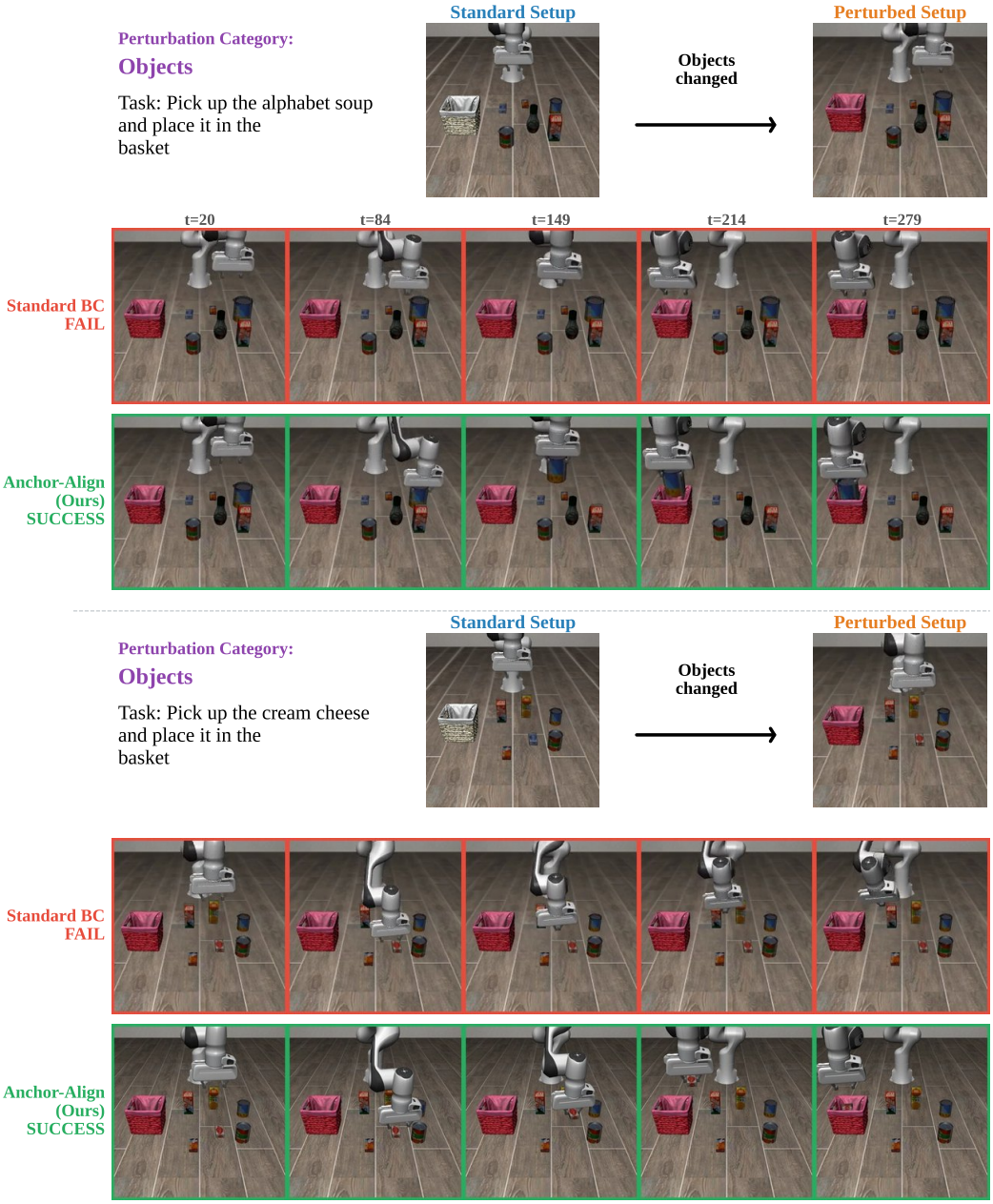


Figure 29: **LIBERO-PRO *Object Swap*.** Both rows use the same object-swap regime: the basket is replaced with a pink wicker variant and the surrounding canned items are exchanged for novel objects unseen during training. *Top:* "Pick up the alphabet soup and place it in the basket": Standard BC (red) hovers over an incorrect target and then moves to the basket without grasping any object, whereas Anchor-Align VLA (green) grounds to the alphabet-soup can and deposits it in the basket. *Bottom:* "Pick up the cream cheese and place it in the basket": Standard BC correctly identifies the target object but fails to grasp it, completing its trajectory with an empty gripper, whereas Anchor-Align VLA selects the cream-cheese box and places it correctly.